Resolve.

DOCUMENTATION LIBRARY / 08

Operations and integrations

Merchant workflows and provider tooling

Operating procedures, private customer flows, provider configuration, Postman simulation instructions and the retained application inventory.

COMPILATION DATE

12 November 2026

Source chapters retain their recorded dates and validation scope. Local, simulated and account-backed evidence are distinguished. This volume does not report a new production acceptance run.

Reading map

This volume brings together the maintained source documents listed below. Source chapters retain their recorded evidence dates and assumptions. Later corrections in the engineering log take precedence over older snapshots.

Chapter	Source in the project
Merchant operations	docs/PLATFORM_EXPANSION.md
Customer portal	docs/CUSTOMER_PORTAL.md
Customer workflow	docs/CUSTOMER_WORKFLOW.md
Hosted follow-up	docs/HOSTED_FOLLOWUP.md
Commerce integration	docs/COMMERCE_INTEGRATION.md
Performance and preview	docs/PERFORMANCE.md
Postman contracts and simulations	postman/README.md
Application inventory	docs/testing/application-inventory.md
Practical merchant and buyer guide	docs/RESOLVE_GUIDE.md

Merchant operations

Source: docs/PLATFORM_EXPANSION.md

Merchant operations expansion

Implementation scope for CHG-048, 2026-11-10. All endpoints retain merchant ownership and separate refund approval/execution. Provider access is opt-in; local test evidence does not establish account-backed success.

Implemented workflows

1. `POST /platform/account/signup`, `/login`, `/accept-invite` create individual

password-backed sessions. Owners invite staff and change roles at `/platform/team`. Roles: owner, manager (refund approvals), finance (refund execution), agent (case work), viewer (read only). Existing full merchant keys retain administrator authority; scoped commerce keys retain their restrictions.

2. `/platform/connections/{provider}` saves encrypted, merchant-scoped credentials

and exposes only redacted configuration/status. Reads check the exact provider hostname. PayPal Sandbox writes still need explicit enablement and approvals.

3. `/platform/imports` imports Zendesk ticket comments; signed webhook deliveries

enqueue ticket IDs for a separate worker. Exact order-reference matching may link a case; ambiguous/no matches remain in a merchant review queue. Imported private comments stay private. Source IDs deduplicate retries and feed the existing optional extraction flow. Suggestions never become promises silently.

4. `/platform/watchdog` scans the whole owned workspace, snapshots overdue current

promises, identifies pending/failed/missing proof, assigns the configured case owner label (assigning unassigned cases to an active workspace owner) and prepares customer-update drafts. Daily repeat scans deduplicate. It does not auto-send updates or issue refunds.

5. `/platform/cases/{id}/offers` publishes versioned options referencing exact

promise revisions. Buyer capability routes list safe terms and record one version-bound acceptance/decline. Stale, expired, replaced and revoked-link attempts fail. A published refund offer adds an acceptance gate to payment.

6. `/platform/dashboard` aggregates the whole merchant workspace. USD financial

totals deduplicate shared captures, separate completed/pending/outstanding, and do not treat absent provider history as a verified zero. Case closure metrics report only known timestamps and include their sample count.

7. `/platform/cases/{id}/returns` guides item selection from registered orders,

quantities, reasons, merchant approval/rejection, return tracking, warehouse receipt/inspection and replacement shipment. Updates use revision checks. Carrier delivery and internal receipt remain separate evidence. Physical shipment/label purchase and Shopify mutation permissions are external steps.

8. `/platform/cases/{id}/dispute-packages` creates immutable, hashed evidence

packages with payment evidence, conversation, promises, source labels, attachments and missing-evidence checklists. A merchant reviews then exports. Provider dispute submission is a later explicitly approved integration, as in the requested roadmap; generating a package does not submit it to PayPal.

Validation boundaries

Use only disposable SQLite/PostgreSQL for migration and authorization tests. Exercise real application paths with HTTP stand-ins for provider pagination, authentication, retries and outages. Validate stale offer acceptance, cross-tenant denial, role revocation, financial permission boundaries, shared-capture precision, duplicate imports/alerts/returns and immutable package history. Regenerate the FastAPI OpenAPI contract and SDK types after implementation. Verify the native merchant and buyer UI separately from API tests.

Start using it

1. Start the API and frontend using their existing README commands. Open

`http://127.0.0.1:3000/account` to create a new workspace. To keep an existing workspace, sign in with its full merchant key at `/login`, then open `/account?claim=1` to create its first owner account. Signup is controlled by `RESOLVE_MERCHANT_SIGNUP_ENABLED` (default true).

2. Open `/workspace/operations`. The Overview shows case outcomes, evidence

gaps, overdue promises, complaint categories and closure times. Financial totals are USD recorded-ledger amounts, separate from independently verified PayPal evidence. Shared captures count once; promises are cumulative amounts, and pending amounts are separate from completed refunds.

3. In Team, create an invitation and privately share its link. It expires after

two days and can be used once by the specified email. Invitations are not emailed automatically. Changing role or disabling a member revokes their sessions. At least one active owner must remain. A staff email currently belongs to one workspace.

4. In Connections, save only the provider credentials needed for this workspace,

explicitly enable the connection, then choose Check connection. Empty secret fields retain existing credentials. Set a stable Fernet `RESOLVE_PLATFORM_ENCRYPTION_KEY` in the API and worker environments before saving connections. `.env.example` has the generation command; the prepared ignored local file already has a generated key. Keep it with private backups. Changing a connection requires fresh financial review. A successful PayPal credential check does not prove the capture belongs to the expected seller.

5. In Support imports, enter a Zendesk ticket ID. Exact ticket `external_id`

matches a unique case/order reference; a unique registered order can open its case automatically. Otherwise, select and confirm the correct merchant case. Imported comments preserve IDs, quotes, author roles, timestamps and public/ private labels. They remain merchant evidence and are not copied into the buyer conversation. The existing extraction panel can propose promises from these sources when AI is enabled and policy permits; review them before saving.

6. Configure a Zendesk ticket trigger with the displayed endpoint:

`https://YOUR_FRONTEND/webhooks/zendesk/MERCHANT_UUID`. Save that webhook's signing secret in its owning Zendesk connection, not an API token in its place. Use the payload below, and retain Zendesk's signature and signature-timestamp headers. The invocation ID header is preferred when present. Public HTTPS delivery requires a hosted frontend or a separately configured tunnel.

```
{"ticket_id":"{{ticket.id}}","event_id":"{{ticket.id}}:{{ticket.updated_at}}"}
```

7. From `backend`, run `uv run python -m app.operations_worker --loop`. It reads

the ordinary backend settings, decrypts owning workspace connections, handles bounded imports with leases/retries, and scans overdue commitments every minute. Alerts deduplicate by promise revision and UTC day. Use Missed promises -> Scan workspace for an immediate pass; copy and review customer drafts. The worker neither sends mail nor authorizes/executes payments. Existing investigation, follow-up and refund-recovery workers remain separate.

8. Open a case to publish up to four remedy choices from current promise

revisions, with an expiry within thirty days. Share the existing private buyer link. Buyers see safe terms and remaining conditions and can accept one choice or decline. Replaced/revised/expired offers and revoked links cannot be accepted. A refund offer requires the selected current acceptance before financial approval; all existing condition, amount, provider-history and explicit execution gates still apply.

9. Sync item-level orders through the commerce API/SDK to enable guided returns.

Buyers or agents choose products and quantities, then a merchant approves or rejects. Merchant controls record tracking, warehouse receipt, inspection, replacement tracking and completion in order. Reserved quantities prevent duplicate over-returns across cases for the same registered order. Rejected requests release quantities. Optional exact-revision milestone confirmation records merchant evidence for a matching return/replacement promise. It does not claim a carrier independently verified the milestone.

10. In the case's dispute panel, enter the dispute reference and reason, prepare

a package, inspect its evidence/missing-evidence checklist, and have an owner or manager review it before export. The JSON export contains the full case timeline, payment evidence, private source conversations, and attachment metadata/hashes; raw attachment files remain separate. Keep exported private evidence secure. Evidence changes require a newly prepared/reviewed package; historical snapshots remain readable. Nothing is submitted to PayPal.

Permissions

Role	Case work / offers / returns	Approve refunds	Execute approved Sandbox refunds	Review dispute packages	Team / connections / SDK keys
Owner	Yes	Yes	Yes	Yes	Yes
Manager	Yes	Yes	No	Yes	No
Finance	Yes	Yes	Yes	No	No
Agent	Yes	No	No	No	No
Viewer	Read	No	No	No	Read

All staff are scoped to one merchant. Legacy full keys and existing key/PayPal browser sessions retain administrator authority; do not distribute full keys to staff. Commerce-only keys cannot access these operations. Authenticated writes record the staff UUID or a credential pseudonym, request intent and completion; this supplements existing domain audit events. Passwords use salted scrypt and opaque hashed sessions; login/signup are throttled. MFA, verified email, password reset/recovery, cross-workspace memberships and self-service OAuth installation are follow-up work. Current onboarding accepts merchant-provided provider tokens.

Storage, contracts and checks

SQLite migration 026 and PostgreSQL migrations 016-017 add identities, encrypted connections, import jobs/sources, offers, alerts, returns and package snapshots. 017 makes new foreign keys deferrable for the existing SQLite import order; 016 was retained unchanged. API startup applies migrations. Back up storage and the encryption key before operating a hosted deployment. No prior migration is rewritten. OpenAPI is generated at `postman/contracts/resolve.openapi.json`; TypeScript operations helpers use those types with a full key and direct API URL.

Validation for CHG-048 used synthetic data: full backend regression, disposable PostgreSQL upgrade/import and operations tests, local signed-webhook/provider failure tests, a real Postman TLS proxy (eight mocked requests, zero pass-through), packaged SDK integration tests, frontend type/lint checks and production build, and a desktop/mobile owner -> buyer -> invited staff browser workflow. See CHG-048 (ENGINEERING_LOG.md) for exact commands, results and limitations. No real Zendesk delivery, new provider account authorization, Shopify return/ replacement mutation, email delivery, carrier label purchase or PayPal dispute submission was exercised. Existing Sandbox-only financial execution is retained.

Customer portal

Source: docs/CUSTOMER_PORTAL.md

Customer portal

To start a new case from an order-specific link, see customer intake and case triage (CUSTOMER_WORKFLOW.md).

Resolve has two views of the same resolution. A merchant works in the inbox. A customer opens a case link to see the customer-facing promise, progress, and conversation. The customer does not need a merchant account.

Route	Access	Purpose
`GET /resolution-cases/{id}/customer-link`	Owning merchant	See whether a link is active and when it expires.
`POST /resolution-cases/{id}/customer-link`	Owning merchant	Create or replace the link and choose a public heading and whether to display the order reference. Returns the new token once.
`DELETE /resolution-cases/{id}/customer-link`	Owning merchant	Revoke access immediately.
`GET /resolution-cases/{id}/customer-messages`	Owning merchant	Read the case conversation.
`POST /resolution-cases/{id}/customer-messages`	Owning merchant	Reply in the portal; cases with an email contact queue a separate notice.
`POST /customer-portal/view`	Customer link token	Read the limited public case projection and conversation.
`POST /customer-portal/messages`	Customer link token	Send a message with a request ID for safe retries.

The link expires after seven days. Replacing or revoking it invalidates the old token. A link grants access to the current customer-facing promises and case conversation, so send it only to the intended customer. The private merchant title, source notes, attachments, evidence export, payment IDs, approval records, and internal review reasons are not returned.

To try the customer screen without an account, open the sample portal (http://127.0.0.1:3000/customer?demo=1). For an actual local case, connect to FastAPI in the merchant UI and open its **Customer portal** tab. Choose the public heading, optionally show the order reference, and select **Create customer link**. Copy the link or open the customer view in a separate browser session. The merchant and customer can reply in the case; refresh messages to see updates.

The browser keeps the token in the URL fragment (https://developer.mozilla.org/en-US/docs/Web/URI/Reference/Fragment) so it is not part of the initial page request. API requests carry it in their JSON body and return private, non-cacheable responses. The database stores its hash. The full link is shown only when created; replacing it is how to obtain a new copy. Messages are plain text, capped at 2,000 characters and 100 per case. Customer messages are limited to six per minute; reusing the same request ID and text is an idempotent retry. A conversation never changes approval or payment state. Hosted email notices are optional and never include the conversation text. See hosted follow-up (HOSTED_FOLLOWUP.md).

The app runs on loopback during development, so these links work on the machine running Resolve. Use a hosted HTTPS environment before sharing links with customers on other devices. Each link is limited to its assigned case. Buyers can also return through My cases (CUSTOMER_SIGN_IN.md) if their order invite was associated with a verified address.

Customer workflow

Source: docs/CUSTOMER_WORKFLOW.md

Customer intake and case triage

1. Start the backend and frontend as shown in the local UI guide (../frontend/README.md).

Connect a merchant account and choose **Invite customer**. Enter the order reference, copy the private link, and share it with the buyer.

2. Open the link as a buyer in a separate browser session. Choose an issue

(missing item, duplicate charge, damaged delivery, refund delay, or other) and submit a description. The same link now opens the new case.

3. Attach a photo, receipt, or return slip under **Customer evidence**. The

buyer and merchant both see the file labeled **Customer submitted**.

4. Reload the merchant workspace. Search by order, title, case ID, or assignee,

or filter by **Waiting on customer**, **Needs approval**, **Pending with PayPal**, or **Unread**. Open **Manage case** to assign a person or team and set the next deadline. Open **Customer portal** in the case to view the conversation, download evidence, reply, and mark displayed messages read.

5. If the case needs a refund, use **Link payment record** on the existing case,

select the correct merchant-owned payment case, and record the agreed refund promise. This keeps the original issue, conversation, evidence, and buyer URL. The payment record must already exist; use the existing payment API to create one if needed. Linking alone does not request a refund.

Try the standalone sample intake at localhost:3000/customer?intake=1 (http://127.0.0.1:3000/customer?intake=1). It creates no backend records. An ordinary link issued by the local workspace is reachable on this machine. For emailed invites, notifications and remote access, see hosted follow-up setup (HOSTED_FOLLOWUP.md).

The merchant creates an invite for one order. The invite itself authorizes access; entering an order number never reveals a case. The first valid submission creates one case and turns the same invite into a case link for seven days from submission. A retry returns the original case. Replace or revoke the case link in **Customer portal**; replaying the original invite cannot restore access. Share private links only with the intended buyer.

Route	Purpose
`POST /resolution-cases/intake-invites`	Merchant issues a seven-day, order-specific invite.
`POST /customer-portal/intake-view`, `POST /customer-portal/intake`	Read only the invited order reference; submit one issue for that invite.
`POST /customer-portal/evidence`, `POST /customer-portal/evidence/{id}`	Upload and download case-scoped customer evidence.
`GET /resolution-cases/{id}/customer-evidence`, `GET /resolution-cases/{id}/customer-evidence/{file_id}`	List and download customer-submitted evidence as the owning merchant.
`GET /resolution-cases/{id}/operations`, `PUT /resolution-cases/{id}/operations`	Read derived inbox state; update assignee label, due date and waiting-on-customer flag.

Route	Purpose
`POST /resolution-cases/{id}/operations/read`	Mark customer messages read through a displayed watermark.
`GET /resolution-cases/inbox`	Search and filter the merchant's cases before pagination.
`PUT /resolution-cases/{id}/payment-link`	Attach one existing payment case owned by the merchant.

Uploads accept PDF, PNG, and JPEG up to 2 MiB each, with a maximum of 10 files per case. File names, media types, and file signatures are checked. An identical file with the same name can be retried without another attachment. Downloads require an active case link or the owning merchant and are served as attachments. Customer files remain separate from the evidence that verifies a remedy or changes refund eligibility.

The timeline shows the report and the milestones that have evidence, such as **Return confirmed**, **Refund approved**, and **PayPal confirmed a refund**. The source label distinguishes a customer submission, a merchant finding, and provider confirmation. The next actor comes from the same workflow rules as the inbox. A case due date overrides the earliest outstanding promise deadline. Overdue means the chosen deadline passed while work remains. A new buyer reply clears the waiting-on-customer flag and becomes unread. Reading through one message does not clear newer replies. Assignment and read state are shared merchant fields; assignment labels do not grant account permissions.

Hosted follow-up

Source: docs/HOSTED_FOLLOWUP.md

Hosted customer follow-up

The order invite, case, and customer portal remain usable without any external integration. The contract for the optional follow-up integrations is:

Action	Contract
Invite by email	The merchant may supply `customer_email` when creating an order intake invite. The same invite is queued for email; submitting it binds the case to that address.
Reply and refund notices	Queue one message per merchant reply and per independently verified completed PayPal refund on a shared refund promise. Messages link to My cases and omit the conversation and amounts.
My cases	Sign in with a verified Google mailbox or redeem a short-lived email link. List only cases assigned to that address. Open a case through a separate short-lived link; merchant revocation still applies.
Automatic checks	The worker processes already-verified PayPal webhook receipts and periodically refreshes enabled EasyPost and Shopify promises through the existing proof adapters. Carrier and warehouse milestones remain independent.
Helpdesk	A merchant can link an existing Zendesk ticket and refresh its status. Resolve reads the ticket summary; customer documents, messages, and payment data stay in Resolve.

Configure when the customer portal is hosted

Keep FastAPI on the same host as the Next.js server and listen on loopback. Put the customer pages and their narrow API proxies behind HTTPS. Preserve the public `Host` header and expose `/customer`, `/customer/account`, `/api/customer-portal/*`, `/api/customer-account/*` and the Next.js static assets. Keep the general FastAPI API private; to also expose the merchant workspace, follow the merchant sign-in guide (MERCHANT_SIGN_IN.md). Set `RESOLVE_CUSTOMER_PUBLIC_ORIGIN` to the same exact HTTPS origin in both `backend/.env` and `frontend/.env.local`.

Resend is the default email transport. Buyer sign-in and email (CUSTOMER_SIGN_IN.md) covers Google sign-in and the optional Nodemailer SMTP transport.

1. Verify the sending domain in Resend (https://resend.com/docs/dashboard/domains/introduction).

Disable click tracking for the private support and sign-in links. In the backend set `RESOLVE_CUSTOMER_EMAIL_ENABLED=true`, `RESOLVE_CUSTOMER_EMAIL_MERCHANT_ID` to the owning Resolve merchant UUID, `RESOLVE_RESEND_FROM` to a sender on that domain and `RESOLVE_RESEND_API_KEY` to a sending key. Set `RESOLVE_CUSTOMER_SIGNING_KEY` to a stable random secret (at least 32 characters). Share it with the worker through its environment. Keep this key and the public origin stable while email is queued.

2. Start the bounded worker from `backend`: `uv run python -m app.followup_worker --loop`.

It prints counts, never addresses, tokens or private message contents. The queue distinguishes provider acceptance from mailbox delivery. Retryable errors use the same Resend idempotency key (https://resend.com/docs/dashboard/emails/idempotency-keys) within the provider's 24-hour window; stale jobs are not resent. The worker does not initiate, approve, or retry refunds. The existing refund recovery worker is separate.

3. In the merchant inbox choose **Invite customer**, enter the order reference

and the buyer email already verified by the merchant for that order. Select **Email this invite from Resolve** to send it. Either way, entering an email binds the resulting case to that address. Copy-only invites do not send automatic notifications. **My**

cases is a separate customer sign-in screen; it does not accept a merchant key. Sign-in links expire in 15 minutes and can be used once. The account session lasts up to seven days; links opened from it last at most one hour. An expired original share link does not hide an account case. Merchant revocation removes it from My cases and invalidates links already opened from that account.

Provider updates

1. Register a stable public HTTPS callback on the **same PayPal Sandbox REST

app** used for the order. Route only `POST /webhooks/paypal/sandbox` to FastAPI and preserve the raw body and PayPal signature headers. Set `RESOLVE_PAYPAL_SANDBOX_WEBHOOK_ID` to the registered ID and use the existing supported events from PayPal reconciliation (PAYPAL_RECONCILIATION.md).

2. Confirm a genuine Sandbox delivery appears as verified in Resolve. PayPal's

integration guide (https://developer.paypal.com/api/rest/webhooks/rest/) explains subscription and verification. Simulated signatures prove the local handler and worker path; they do not prove this registration or a real callback. To validate without money movement, a temporary subscription to `INVOICING.INVOICE.CREATED` can receive an unsent Sandbox draft invoice. A verified irrelevant event is stored as `ignored`. A supported refund event is required to validate account-backed reconciliation. To inspect recent receipts without dumping event bodies or signature headers, run this from `backend`:

```
@'
from app.settings import Settings
from app.storage import configured_store
config = Settings()
with configured_store(config).transaction() as db:
    for row in db.execute(
        """SELECT event_type,status,received_at FROM paypal_webhook_events
           WHERE merchant_id=? ORDER BY received_at DESC LIMIT 5""",
        (str(config.paypal_resolve_merchant_id),),
    ):
        print(*row)
'@ | uv run python -
```

3. Set `RESOLVE_AUTO_PROVIDER_REFRESH_ENABLED=true` with the merchant-scoped

EasyPost and/or Shopify credentials in `backend/.env`. Unresolved provider promises are checked about every five minutes and fulfilled ones about hourly. The worker checks replacements, carrier-delivered returns, Shopify cancellations and tracked exchange fulfillment. Unchanged observations do not create new timeline entries. Warehouse receipt and inspection still require an explicit merchant finding. These are scheduled API reads, so EasyPost and Shopify do not need incoming webhook registration.

Link an existing helpdesk ticket

Set `RESOLVE_HELPDESK_MERCHANT_ID`, `RESOLVE_ZENDESK_SUBDOMAIN`, and `RESOLVE_ZENDESK_OAUTH_TOKEN` with access to read Zendesk tickets (https://developer.zendesk.com/api-reference/ticketing/tickets/tickets/#show-ticket). Replace the OAuth access token when it expires; Resolve does not refresh it. Open the resolution case and use the **Zendesk ticket** card to link a ticket ID, refresh the status, or open the ticket in Zendesk. Resolve stores the ticket ID, status, and last check. It does not create Zendesk tickets, copy comments, or send evidence to Zendesk. Zendesk credentials are never exposed to the buyer or browser.

The worker and backend use the same database settings. For shared hosted storage, see PostgreSQL migration (POSTGRES_MIGRATION.md). A local SQLite file is for a single host. API contracts are in the generated OpenAPI spec (../postman/contracts/resolve.openapi.json).

Commerce integration

Source: docs/COMMERCE_INTEGRATION.md

Connect a merchant app to Resolve

Use this integration when your application already accepts PayPal and you want support staff to see the app's order and PayPal's evidence in the same Resolve case. Start by sharing one selected order; the SDK does not scan or backfill your entire database.

Show all orders in the merchant dashboard

Sign in to your regular Resolve merchant workspace and select **Orders**. The dashboard lists the orders your apps have registered for that merchant, including unpaid orders and orders without a support case. Search by store order reference or PayPal order/capture ID, or filter by source. Select an order to see the app snapshot. Use **Create resolution case** only when the order needs investigation; it keeps the same order and opens the existing case workspace with the evidence attached.

Point your app at the **same Resolve API and merchant** as the dashboard. Use `registerCommerceOrder(snapshot)` for every order, then call it again after changes to the order, payment, or line items. A revision is an integer that increases when those fields change. Use one stable source name for both backfill and ongoing events so historical and new snapshots update the same order.

For existing PostgreSQL orders, the adapter reads the authorized orders view in batches. Start ongoing sync before backfill so concurrent app changes are also sent. Save the checkpoint after each successful registration:

```
const orders = resolve.connectPostgres(readOnlyPool, { source: "my-store" });
for await (const receipt of orders.registerAll({ after: savedCheckpoint, pageSize: 50 })) {
  await saveCheckpoint(receipt.order_reference);
}
```

Run the import example (../sdk/typescript/examples/import-orders.mjs) with the same environment as the PostgreSQL example below and optionally set `RESOLVE_APP_SOURCE` and `RESOLVE_IMPORT_AFTER`. If an order changes during backfill, a stale revision can be rejected; read the latest row and retry, or resume after a checkpoint only when that order has already been synced. For other databases, page through your own orders and pass each snapshot to `registerCommerceOrder` or `POST /commerce/orders/register`. Resolve doesn't pull a merchant's PayPal history or connect to the app's database on its own.

The demo store's isolated launcher uses a separate merchant and database. It does not populate the regular dashboard. To connect Field Brew to the regular workspace, use its external workspace setup (../examples/checkout-store/README.md#connect-it-to-a-different-resolve-workspace). An existing Sandbox purchase can be registered with its original PayPal IDs without approving or capturing a new payment.

Start here

1. Sign in with your Resolve access key and open **Settings -> SDK keys**.

Select **Generate SDK key** and **Copy SDK setup**. Use a new access-key sign-in if Resolve asks you to reauthenticate. A generated key is displayed once. The same screen lists active key IDs and lets you revoke them.

2. Paste the copied `RESOLVE_API_URL`, `RESOLVE_API_KEY`, and optional

`RESOLVE_APP_SOURCE` into your app server's environment. Use a different stable source per app. Never include the key in browser code.

3. Install the Node.js SDK archive (../sdk/typescript/README.md#install-this-preview)

or call the direct HTTP endpoints. An SDK key cannot sign in, read merchant contacts, open cases from the dashboard, or execute a payment.

```
import { createResolve } from "@resolve-paypal/sdk";

await createResolve().syncOrder({ id: "STORE-2042", status: "paid" });
```

This publishes revision 1 in **Orders**. Send the same ID with a higher `version` when the order changes. This call doesn't need a PayPal connection; add the app's payment fields and configure PayPal Sandbox reads (../backend/README.md#paypal-sandbox-capture-reads) if you want to verify the payment. Refund write flags are unnecessary.

The copied URL ends in `/api/sdk` on your Resolve web app. That gateway accepts a bearer key and exposes commerce registration, case-first sync, and evidence read/refresh only. For other backend operations, use your configured FastAPI URL directly. Local administrators can still issue a commerce key without the dashboard:

```
uv --directory backend run python -m app.manage issue-key --merchant-id YOUR_MERCHANT_UUID --scope commerce
```

Option 1: Read PostgreSQL from your Node server

Apply the view and grant template (../sdk/typescript/examples/postgres-views.sql) in your **application database** after mapping your tables. Use an app-owned database login that inherits only the relevant view grants. The SDK receives a pool, not your schema, password, or raw tables; the database connection stays inside your Node server. Do not reuse an admin, application writer, or shared role with broader grants.

App evidence view	Required columns and mapping
`resolve_evidence.orders`	`order_reference` (text), `revision` (positive integer), `order_status` (text or null), `customer_reference` (opaque ID or null). One row for the chosen order.
`resolve_evidence.payments`	`order_reference`, `paypal_order_id`, `paypal_capture_id`, `currency`, normalized `status`, `captured_amount`, `refunded_amount`. Export at most one PayPal USD payment per order. An order can be synced before payment.
`resolve_evidence.items`	`order_reference`, stable sortable `line_id`, `product_id`, `name`, `sku`, `quantity`, `unit_amount`. Export the order's line quantity and historical price; name and SKU may be null. At most 50 items.
`resolve_evidence.contacts`	**Optional**: `order_reference` and the approved columns from `name`, `email`, and phone. Optional view template (../sdk/typescript/examples/postgres-contact-view.sql). No contact view is queried by default.

Use decimal amounts in USD, not integer cents, in the views. The SDK reads money as text so amounts do not pass through JavaScript floating point. Use payment statuses `pending`, `completed`, `partially_refunded`, `refunded`, or `failed`. Use `null` for unknown amounts or status, and use `0.00` only when you know the total is zero. `refunded_amount` means the sum of **completed** refunds. Use a stable app revision and increment it in the same app transaction as any change to the exported order, payment, items, or shared contact fields. If changing which contact fields you share, increment the affected order's revision before re-syncing. In a shared database, apply the correct merchant row filter to every view; the SDK role must only be able to read that merchant's data. The views use PostgreSQL view permissions (https://www.postgresql.org/docs/18/sql-createview.html) with explicit SELECT grants.

```
import { Pool } from "pg";
import { ResolveClient } from "@resolve-paypal/sdk";

const resolve = new ResolveClient({
  baseUrl: process.env.RESOLVE_API_URL!,
  apiKey: process.env.RESOLVE_API_KEY!,
  timeoutMs: 30_000,
});
const pool = new Pool({
  connectionString: process.env.APP_EVIDENCE_DATABASE_URL!,
  connectionTimeoutMillis: 5_000,
```

```
  statement_timeout: 5_000,
});
const evidence = resolve.connectPostgres(pool, { source: "my-store" });

// Check that the caller is authorized for this order in your app first.
const report = await evidence.inspect("STORE-2042");
```

Reuse the pool and client in your server. `evidence.inspect(orderId)` reads one order, syncs it, and refreshes the linked PayPal evidence. Use `evidence.sync(orderId)` for an update that makes no PayPal request. Either can take `{ caseId }` to attach an existing owned Resolve case with the same order reference (or no order reference). The pool only receives a parameterized query (https://node-postgres.com/features/queries#parameterized-query) for that order. The app database and Resolve database do not need to be the same engine or reachable from each other.

For an end-to-end starter, run connect-postgres.mjs (../sdk/typescript/examples/connect-postgres.mjs) with your read-only app database URL and order ID. Install `pg` in the app and run from its project root:

```
npm install pg
node node_modules/@resolve-paypal/sdk/examples/connect-postgres.mjs STORE-2042
```

Option 2: Push a snapshot from any backend

Use `resolve.syncCommerceOrder(payload)` or call the API from your existing server, for example after an order/payment changes or when a support case opens.

```
{
  "source": "my-store",
  "order_reference": "STORE-2042",
  "revision": 1,
  "order_status": "shipped",
  "customer_reference": "CUSTOMER-42",
  "payment": {
    "paypal_order_id": "EXAMPLEPAYPALORDER",
    "paypal_capture_id": "EXAMPLEPAYPALCAPTURE",
    "currency": "USD",
    "status": "completed",
    "captured_amount": "100.00",
    "refunded_amount": "0.00"
  },
  "items": [{"product_id": "PRODUCT-42", "name": "Desk lamp", "quantity": 2, "unit_amount": "50.00"}]
}

$headers = @{ Authorization = "Bearer $env:RESOLVE_API_KEY" }
$sync = Invoke-RestMethod -Method Post -Uri "$env:RESOLVE_API_URL/commerce/orders/sync" -Headers $headers -ContentType
"application/json" -InFile "order.json"
Invoke-RestMethod -Method Post -Uri "$env:RESOLVE_API_URL/resolution-cases/$($sync.case_id)/commerce/refresh" -Headers
$headers
```

Endpoint	Purpose
`POST /commerce/orders/register`	Register or update an order using a commerce key. Returns its stable Resolve order ID and optional case ID.
`GET /commerce/orders` / `GET /commerce/orders/{order_id}`	Full merchant access: browse, search and read app snapshots without PayPal calls or shared contact fields.
`POST /commerce/orders/{order_id}/case`	Full merchant access: create or reuse the order's case and link evidence.
`POST /commerce/orders/sync`	Create or reuse a case by merchant, source and app order ID. Save the latest snapshot and link PayPal IDs in one transaction. Optional `case_id` attaches an existing owned case.
`GET /resolution-cases/{case_id}/commerce`	Read the current snapshot and comparison, without making a PayPal request.

Endpoint	Purpose
`POST /resolution-cases/{case_id}/commerce/refresh`	Check linked PayPal Sandbox evidence and return the refreshed comparison.
`GET /resolution-cases/{case_id}/commerce/contact`	Read approved contact fields if present. Requires full merchant access.
`GET /resolution-cases/{case_id}/evidence-bundle`	Full merchant access: return the existing timeline and verified evidence plus the app comparison, without contact fields.

Identical requests for the current revision reuse the receipt, including concurrent requests. A changed payload at the same revision, a stale revision, a conflicting case binding, or replacement/removal of an existing PayPal reference returns `409`. A capture can be added later. Established payment IDs stay attached to their case so a correction cannot silently rewrite the identity of previously recorded evidence. Check IDs before the first sync. The endpoint accepts at most 32 KiB and rejects unknown fields. See the OpenAPI contract (../postman/contracts/resolve.openapi.json) for types.

What the comparison means

Resolve records the app's snapshot as merchant-reported evidence. It compares PayPal order and capture identifiers, currency, gross captured amount, payment status, and the completed refund total when it can reconcile the order. It reports `match`, `mismatch`, or `incomplete`, with app and provider values where known and a check time per verified field. `missing_app` means PayPal confirmed a fact but the app did not supply it. `unverified` means Resolve lacks applicable provider evidence. A newer app revision needs a check started after its sync, even if an older check finished later. An outage leaves the earlier PayPal facts in the evidence bundle, but the current comparison won't present those facts as freshly confirmed. Disagreement never changes either the app's tables or PayPal.

The current provider scope is the configured seller's PayPal Sandbox, one USD capture per order. Add the actual PayPal order ID if the capture does not identify it. An unsupported or incomplete history is reported explicitly; it cannot establish zero refunds. Product and customer details remain app records, not independently verified PayPal facts. The integration does not onboard arbitrary sellers or pull cross-app or live payment history.

Contact and credentials

By default, only an opaque customer reference is read. To share contact data intentionally, select just the approved fields in the PostgreSQL grant and `contactFields: ["email"]` (or submit those fields in the direct API's `contact` object). Resolve keeps only the latest contact data on that case. An updated revision that omits `contact` removes it from the app-facing record. It is not included in the comparison, evidence bundle, timeline, buyer view, or automatic messages. It does not enroll a buyer in My cases. No password, password hash, auth token, card data, or full user row belongs in the integration.

Performance and preview

Source: docs/PERFORMANCE.md

Responsive local preview and case loading

CHG-051, 2026-11-10.

Run the optimized local app

Start the API from `backend`:

```
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000
```

In another terminal at the repository root:

```
npm --prefix frontend run build:preview
npm --prefix frontend run preview
```

Open http://127.0.0.1:3000/workspace. The compiled preview uses `.next-preview` and the existing local frontend configuration. The script binds loopback only; it does not publish the website. Stop any frontend already using port 3000 first. Source/configuration changes require rebuilding and restarting this preview. Use `npm run dev` inside frontend for automatic rebuilds while editing.

What changed

• Case details appear before their linked payment requests complete. Until the

current payment and reconciliation are available, financial controls remain ineligible. Backend approval/submission checks remain authoritative.

• The evidence timeline loads on first visit and is reused within that case.

Changing cases or refreshing details invalidates that local timeline. Late responses from a previously selected case cannot replace the current detail.

• Closed offers/returns/packages, return guides, dispute tools, Zendesk and

investigations no longer all fetch immediately. The original connected-app request still loads in the overview. Investigations stop polling when closed.

• The customer tab loads independent reads together and retains its mounted state

when switching tabs in the same case, preserving an unfinished heading draft. Use its refresh action for newly arrived messages. Changing cases resets state.

• Operations loads identity once per mount and runs each tab's independent reads

together. Obsolete responses cannot overwrite the newly selected tab's state.

• Narrow merchant layouts wrap the header/actions and let the case tab strip

scroll locally, rather than widening the whole page.

• PostgreSQL reuses connections through psycopg-pool 3.3.3: lazy opening, maximum

eight connections per process/store, at most 32 queued borrowers, an eight-second checkout deadline, 60-second excess-idle and ten-minute lifetime limits. No financial operation is automatically replayed after a connection failure. Broken connections are discarded; transaction-local schema/read settings reset and the API closes resources at shutdown. Reads retain repeatable-read/read-only isolation; write locks and transaction boundaries remain.

• Authenticated browser reads resolve identity once per request, using the same

transaction for legacy/staff fallback and a joined merchant lookup. Writes recheck the current session at the authorization gate. Sessions, roles and financial results are not cached across requests.

Evidence and limits

A read-only `SELECT 1` against the configured hosted PostgreSQL database took 3.330, 3.359 and 3.182 seconds with fresh connections. After pooling and combining read setup, the cold read took 2.939 seconds; warm reads took 1.211 and 1.182 seconds. These are local connectivity measurements, not whole-page or provider timings. Network distance still affects hosted database reads.

The CHG-051 isolated production browser check observed five distinct initial API paths for a simple case, no unopened secondary-panel requests, one timeline request across a return visit and no customer-panel refetch on a tab return. It checked rapid case changes and a 390px viewport. The complete merchant/buyer workflow also passed against this compiled build. Synthetic SQLite/PostgreSQL tests verify rollback, pooled connection reuse, read-only/schema isolation, broken connection recovery and permission revocation. See the engineering log for exact commands, counts and unsupported SQLite-specific fixtures in the exploratory PostgreSQL run.

Run the compiled synthetic browser checks from frontend with:

```
$env:RESOLVE_UI_PRODUCTION_TEST='1'
npx playwright test -c playwright.platform.config.ts
```

This builds a disposable source copy without owner dotenv files and starts its own synthetic API. Development Strict Mode may make duplicate effect calls; the production check measures the compiled path used by the local preview.

No migrations, API payloads, provider behavior or financial permissions changed. The application has not been deployed or load-tested at production traffic scale. The pool bounds apply per process, so size the total deployment's connection budget before increasing worker/replica counts.

Pool lifecycle follows the Psycopg connection pool documentation (https://www.psycopg.org/psycopg3/docs/advanced/pool.html).

After merge repair CHG-053, the initial case check observes eight distinct API paths: the incoming promise-health, next-action and inbox-risk reads add three paths. Unopened secondary panels remain deferred; timeline reuse, draft retention and mobile overflow checks still pass. The new layout's recent-activity area invites opening the evidence timeline until those events have been loaded.

CHG-056 bounded load and query reduction

The current inbox batches operations/message-count/intake reads for at most 100 cases at a time, within the same transaction. The dashboard groups complaint categories once. No session, role, approval or financial result is cached across requests. A 105-case test checks query growth, filtered pagination totals and equality with ordinary projections; SQLite and actual PostgreSQL checks pass.

The five-route local workload now issues 220 SQL statements per 20 requests, versus 428 before this change. At 1/10/50/100 closed-loop users, all 3,220 final requests returned 200. Measured P95 values were 9.086/97.620/820.103/1702.320ms. Host activity differed between runs; these are bounded local observations, not a controlled speedup percentage or production capacity. See full workload, CPU/RSS and limitations (testing/performance-report.md).

Postman contracts and simulations

Source: postman/README.md

Local PayPal capture and refund simulation

Automatic investigations add generated list/review routes to `contracts/resolve.openapi.json`. Their isolated evaluation uses the real provider adapters with loopback HTTP stand-ins in `backend/app/synthetic_investigations.py`, plus a bounded PayPal read simulation in `backend/tests/test_investigations.py`. Run `uv run python scripts/isolated.py -m pytest tests/test_investigations.py -q` from `backend`. This path needs neither Postman authentication nor real provider keys. It proves local contracts and guardrails, not live provider/model quality. Existing financial simulations below retain their original scope. See model routing and evaluation (../docs/AI_INVESTIGATIONS.md).

The hosted follow-up simulation (simulations/hosted-followup/hosted-followup.sim.yaml) adds Resend and Zendesk to the existing proof adapters. From the repository root:

```
$env:RESOLVE_RUN_FOLLOWUP_SIMULATION = '1'
$env:RESOLVE_RUN_PAYPAL_SIMULATION = '1'
postman simulation run postman/simulations/hosted-followup/hosted-followup.sim.yaml --proxy --lift "uv --directory backend
run python -m pytest tests/test_hosted_followup_simulation.py -q" --output ndjson
```

The follow-up context (simulations/hosted-followup/hosted-followup.context.yaml) separates simulated email and evidence from unverified hosted permissions.

The resolution workspace simulation (simulations/resolution-workspace/resolution-workspace.sim.yaml) also runs the PayPal, Shopify, and EasyPost proof adapters with fake credentials. It includes the original PayPal regressions. From the repository root:

```
$env:RESOLVE_RUN_PAYPAL_SIMULATION = '1'
try {
    postman simulation run postman/simulations/resolution-workspace/resolution-workspace.sim.yaml --proxy --lift "uv
--directory backend run python -m pytest tests/test_resolution_simulation.py tests/test_paypal_simulation.py
tests/test_refund_execution_simulation.py tests/test_paypal_history_simulation.py -q" --output ndjson
} finally {
    Remove-Item Env:RESOLVE_RUN_PAYPAL_SIMULATION -ErrorAction SilentlyContinue
}
```

The evidence contract subset (contracts/resolution-evidence.openapi.yaml) describes the extra provider reads. The simulation context (simulations/resolution-workspace/resolution-workspace.context.yaml) separates fixture evidence from account-backed permissions and observations.

Resolve's generated application OpenAPI (contracts/resolve.openapi.json) is kept separately from the upstream PayPal subset. Regenerate it with `uv run python scripts/export_openapi.py` from `backend` after API/model changes.

This directory keeps the upstream contract subset, local stand-in, simulation, and evidence record with the code. The backend itself runs through its actual routes, HTTPX client, OAuth form encoding, TLS, timeouts, and SQLite writes. The stand-in replaces only PayPal's HTTP endpoints.

Run on Windows

Prerequisites: uv with the backend dependencies synced and Postman CLI 1.82.0 (the version tested here). No Postman sign-in, real PayPal credentials, or sandbox account is needed for this local command. Run from the repository root:

```
$env:RESOLVE_RUN_PAYPAL_SIMULATION = '1'
try {
    postman simulation run postman/simulations/paypal-read/paypal-read.sim.yaml --proxy --lift "uv --directory backend run
python -m pytest tests/test_paypal_history_simulation.py tests/test_paypal_simulation.py
tests/test_refund_execution_simulation.py -q" --output ndjson
} finally {
    Remove-Item Env:RESOLVE_RUN_PAYPAL_SIMULATION -ErrorAction SilentlyContinue
}
```

`--lift` supplies its child process with the local proxy and temporary CA certificate and stops the simulation when the tests end. The test fixture injects that proxy into a real HTTPX client and trusts this CA. It retains the client's fixed sandbox URL and does not disable TLS validation. Normal application clients ignore environment proxies and keep `trust_env=False`.

The simulation must show OAuth and capture requests routed to the `paypal` member and **zero passed-through requests**. The timeout fixture deliberately waits longer than the client's timeout; its late mock response can appear after the assertion that Resolve returned `504`.

Resolve Evidence SDK acceptance test

From the repository root, after installing the SDK's build dependencies:

```
$env:RESOLVE_RUN_SDK_EVIDENCE_SIMULATION = '1'
try {
    postman simulation run postman/simulations/paypal-read/paypal-read.sim.yaml --proxy --lift 'npm --prefix
sdk/typescript test' --output ndjson
} finally {
    Remove-Item Env:RESOLVE_RUN_SDK_EVIDENCE_SIMULATION -ErrorAction SilentlyContinue
}
```

The SDK talks to a disposable FastAPI/SQLite instance. The fixture seeds a USD 25 external refund in the PayPal stand-in. The test checks approval and confirmation as separate records, retries, buyer privacy, tenant isolation, and the behavior when the provider goes down. The mock counters verify that this flow did not submit a refund. This does not establish account-backed visibility.

Commerce integration acceptance test

The same SDK simulation can read a real, disposable merchant app database through the example PostgreSQL views. Set `RESOLVE_COMMERCE_TEST_APP_ADMIN_URL` to a loopback PostgreSQL cluster reserved for tests. The test creates and removes its own database and roles. Run the SDK acceptance command above with that additional variable. It asserts that the app role cannot read a password or write to a view, detects app vs PayPal refund differences, and observes no financial POSTs from the commerce flow.

Store checkout acceptance test

The Field Brew example (../examples/checkout-store/README.md#verify-without-a-real-buyer) uses this same PayPal simulation to approve a cart in Chromium, capture once, sync the app order through the Resolve SDK, and detect a simulated outside refund. Both databases are disposable. The approval controls are mock-only.

Files and contract boundary

• Upstream contract subset (contracts/paypal-capture-read.openapi.yaml): OAuth,

capture GET, approved capture-refund POST, refund GET, order history and webhook signature postback. The historical file and simulation names are retained for existing commands.

• Stand-in configuration (mocks/paypal-sandbox/config.yaml) and

handler (mocks/paypal-sandbox/default.js): synthetic credentials, records, missing fields, mismatches, errors, redirect, and delayed response fixtures. Refund fixtures retain request identity and simulated provider state, including a timeout after refund creation. Unmodeled endpoints return `501`; checkout writes are not modeled. `/__resolve_test/state/{capture_id}` is a harness-only counter endpoint and `/__resolve_test/external/{capture_id}` creates simulated external refunds. Neither endpoint is used by the application or sent to PayPal.

• Simulation (simulations/paypal-read/paypal-read.sim.yaml): intercepts

`api-m.sandbox.paypal.com` only.

• Evidence and unresolved assumptions (simulations/paypal-read/paypal-read.context.yaml):

separates documented provider behavior from locally demonstrated caller behavior.

• Integration tests (../backend/tests/test_paypal_simulation.py): real HTTP

requests with fake values, plus resulting API responses and database state.

• Refund HTTP tests (../backend/tests/test_refund_execution_simulation.py): full

execution, replay, restart/recovery with the same request ID, pending refresh, denial, malformed response and parent-link checks.

• History and webhook HTTP tests (../backend/tests/test_paypal_history_simulation.py):

discovery of an external refund, repeated exact-amount partial refunds, hidden records, stale approval, webhook verification and duplicate delivery, reversal hold, and recovery by unique PayPal correlation without another refund POST.

• Configuration checks (../backend/tests/test_paypal_config.py) and

storage checks (../backend/tests/test_paypal_storage.py): default-suite tests for no-network gates, tenant isolation, atomic rollback, and migration compatibility.

• Local execution tests (../backend/tests/test_refund_executions.py) use explicitly

synthetic provider DTOs for transaction failures, concurrency and lease/state invariants. They do not establish HTTP/provider behavior.

• The HTTP refund module also exercises the recovery worker refreshing a known

pending refund while financial writes are disabled. Combined run: 40 tests passed, 276 mocked requests, zero passed through.

The contract was checked against PayPal's OAuth documentation (https://developer.paypal.com/api/rest/authentication/), capture-read reference (https://developer.paypal.com/api/payments/v2/captures-get), and primary Payments v2 specification (https://github.com/paypal/paypal-rest-api-specifications/blob/main/openapi/payments_payment_v2.json). Orbit identified the capture-read operation but supplied only partial support for the fields this caller needs: its saved example omitted payee. The official schema/example includes payee, without guaranteeing it for every response. The client requires `payee.merchant_id` and fails verification when absent.

Buyer Google sign-in simulation

The buyer Google simulation (simulations/customer-google/customer-google.sim.yaml) intercepts the Google token and public signing key endpoints. The tests register locally signed ID tokens with ephemeral keys, require PKCE at code exchange, and verify that invalid claims and replay cannot create a buyer session.

```
$env:RESOLVE_RUN_GOOGLE_SIMULATION = '1'
try {
    postman simulation run postman/simulations/customer-google/customer-google.sim.yaml --proxy --lift 'uv --directory
backend run --no-sync python -m pytest tests/test_customer_google.py -q' --output ndjson
} finally {
    Remove-Item Env:RESOLVE_RUN_GOOGLE_SIMULATION -ErrorAction SilentlyContinue
}
```

The browser spec `customer-google.spec.ts` also uses this simulation with `RESOLVE_UI_GOOGLE_FIXTURE=1`. Its consent page is a browser fixture, so it does not establish that any Google Cloud app is registered. SMTP is tested by a separate loopback TLS fixture because it is not HTTP.

Merchant sign-in simulation

The same local Sandbox mock supports OAuth authorization-code exchange and UserInfo for merchant sign-in (../docs/MERCHANT_SIGN_IN.md). Run the focused tests from the repository root:

```
$env:RESOLVE_RUN_LOGIN_SIMULATION = '1'
```

```
try {
    postman simulation run postman/simulations/paypal-read/paypal-read.sim.yaml --proxy --lift 'uv --directory backend run
--no-sync python -m pytest tests/test_merchant_auth.py -q' --output ndjson
} finally {
    Remove-Item Env:RESOLVE_RUN_LOGIN_SIMULATION -ErrorAction SilentlyContinue
}
```

This exercises the real Resolve HTTP client against the local mock. A browser test also covers linking, sign-out, and PayPal sign-in with a simulated consent redirect. Neither proves a registered Sandbox or live PayPal app is configured.

The CLI was signed out and the repo was not bound to a Postman workspace, so owner-workspace mock search was skipped. This locally built stand-in is not an owner-published PayPal mock. No artifacts were deployed or pushed to a workspace.

What the evidence establishes

The local run demonstrates exact ID, decimal amount, currency and payee checks, sanitized errors, refused redirects, a real timeout, persistence, approval invalidation, and preservation of old evidence after a failed refresh. It does not establish whether the user's app has the required scopes or returns payee, whether any real capture exists, or whether a refund is eligible.

Capture details, complete refund history, and execution eligibility have separate flags. Overall reconciliation remains `verified_with_paypal=false`; manually entered refunds remain manual. See first-time sandbox setup (../backend/README.md#first-time-setup) to move from simulated evidence to an account-backed sandbox check.

Separately, on 2026-11-07 the application's real OAuth request to PayPal Sandbox succeeded with a nonempty Bearer token and positive expiry. This is credential acceptance evidence only, outside the simulation. A subsequent direct application client check (CHG-010) read one actual sandbox capture and matched its ID, USD 1.00 amount, currency and configured seller; observed status was COMPLETED. No Resolve case or verification observation was saved by that check. The authenticated case-route check subsequently passed in CHG-011. CHG-013 completed a real sandbox USD 1.00 full-refund exercise through actual HTTP routes: one financial POST, independent refund GET, REFUNDED capture and fulfilled reconciliation. This is one sample; complete refund history, partial execution and general eligibility remain unimplemented. See execution and recovery (../docs/REFUND_EXECUTION.md).

Application inventory

Source: docs/testing/application-inventory.md

Application inventory

Inspected 2026-11-11. Generated from source and the checked-in OpenAPI contract; inventory is not proof of runtime behavior.

Structure and technology

- `frontend/`: Next.js 16 App Router, React 19, TypeScript, local CSS and Lucide icons. Server-side API gateways keep merchant credentials out of client code.
- `backend/app/`: FastAPI/Pydantic, synchronous storage services, HTTPX provider adapters, four independent worker entry points.
- `backend/migrations/`: 28 numbered SQLite migrations; `postgres/`: 19 numbered PostgreSQL migrations. Initialization applies checksum-tracked migrations; never edit an existing migration.
- `sdk/typescript/`: server-only Node SDK 0.7, generated OpenAPI types, commerce views/consumers. `examples/checkout-store/`: separate illustrative store, outside the Resolve navigation.
- `postman/`: provider contracts, loopback HTTP mocks and simulations. `docs/`: engineering decisions and feature guides. `.local/`: ignored disposable evidence/runtime files.
- Files/evidence and saved labels are database-backed. No object-storage service is implemented. Provider secrets use Fernet encryption; session/key values are hashed at rest.

All application page routes

Route	Source	Access / purpose
`/account`	`frontend/app/account/page.tsx`	Staff login, workspace signup, owner claim (?claim=1), invitation acceptance (#token)
`/customer/account`	`frontend/app/customer/account/page.tsx`	Buyer My cases; optional email link and Google sign-in
`/customer`	`frontend/app/customer/page.tsx`	Private-link entry, intake, messages, offers, returns; no sample mode
`/demo`	`frontend/app/demo/page.tsx`	Retired route: redirects to /login
`/docs/[slug]`	`frontend/app/docs/[slug]/page.tsx`	Ten public static guide articles, anchors, copy and print
`/docs`	`frontend/app/docs/page.tsx`	Public searchable guide hub
`/login`	`frontend/app/login/page.tsx`	Merchant access-key / optional linked PayPal sign-in
`/`	`frontend/app/page.tsx`	Public landing
`/workspace/operations`	`frontend/app/workspace/operations/page.tsx`	Authenticated overview, missed promises, team, connections, imports and audit
`/workspace/orders`	`frontend/app/workspace/orders/page.tsx`	Authenticated registered orders, search and open/create case
`/workspace`	`frontend/app/workspace/page.tsx`	Authenticated case inbox; ?case=UUID opens nested case details
`/workspace/settings`	`frontend/app/workspace/settings/page.tsx`	Authenticated account, PayPal identity linking and scoped SDK keys
`/docs/getting-started`	`frontend/content/docs.json`	Start your first resolution
`/docs/merchant-workspace`	`frontend/content/docs.json`	Run the merchant workspace
`/docs/buyer-portal`	`frontend/content/docs.json`	Help a buyer without starting over
`/docs/returns-and-replacements`	`frontend/content/docs.json`	Guide returns and replacements
`/docs/refunds-and-disputes`	`frontend/content/docs.json`	Review refunds and disputes
`/docs/sdk-integration`	`frontend/content/docs.json`	Integrate the SDK
`/docs/api-and-reliability`	`frontend/content/docs.json`	Use the API reliably
`/docs/connections`	`frontend/content/docs.json`	Connect supported services
`/docs/running-resolve`	`frontend/content/docs.json`	Run and configure Resolve
`/docs/troubleshooting`	`frontend/content/docs.json`	Troubleshoot common problems

Unknown routes use `app/not-found.tsx`. The API documentation `/docs` on the backend is FastAPI Swagger, distinct from frontend user guides.

Navigation and interactive modes

Case details: Overview, Evidence timeline, Resolution receipt, Customer portal. Inbox includes search, lifecycle/workflow/review/risk filters, new case, invitation and review queue. Overview exposes promises, manual/provider evidence, payment linking, separate financial approval/submission, AI review/extraction and deferred offers/returns/dispute panels. Operations uses six tabs without separate URLs. All are included in the manual and test matrix.

Literal navigation targets (including nested links)

- `#access`: `frontend/app/page.tsx`
- `#docs-content`: `frontend/app/docs/layout.tsx`
- `#main-content`: `frontend/app/page.tsx`
- `#orders-content`: `frontend/components/merchant-orders.tsx`
- `#workflow`: `frontend/app/page.tsx`
- `#workspace-content`: `frontend/components/workspace.tsx`
- `/`: `frontend/app/docs/layout.tsx`, `frontend/app/not-found.tsx`, `frontend/app/page.tsx`, `frontend/components/customer-account.tsx`, `frontend/components/customer-view.tsx`, `frontend/components/merchant-login.tsx`, `frontend/components/merchant-orders.tsx`, `frontend/components/operations-console.tsx`, `frontend/components/workspace-account.tsx`, `frontend/components/workspace-shell.tsx`, `frontend/components/workspace.tsx`
- `/account`: `frontend/app/docs/layout.tsx`, `frontend/app/page.tsx`, `frontend/components/merchant-login.tsx`
- `/account?claim=1`: `frontend/components/operations-console.tsx`
- `/customer`: `frontend/app/not-found.tsx`, `frontend/app/page.tsx`, `frontend/components/customer-account.tsx`, `frontend/components/merchant-login.tsx`
- `/customer/account`: `frontend/app/page.tsx`, `frontend/components/customer-view.tsx`
- `/docs`: `frontend/app/docs/[slug]/page.tsx`, `frontend/app/docs/layout.tsx`, `frontend/app/page.tsx`, `frontend/components/docs-navigation.tsx`, `frontend/components/forms.tsx`, `frontend/components/merchant-login.tsx`, `frontend/components/operations-console.tsx`, `frontend/components/workspace-shell.tsx`, `frontend/components/workspace.tsx`
- `/docs/getting-started`: `frontend/app/docs/layout.tsx`, `frontend/app/docs/page.tsx`
- `/docs/resolve-guide.md`: `frontend/app/docs/[slug]/page.tsx`, `frontend/app/docs/page.tsx`, `frontend/components/docs-navigation.tsx`
- `/docs/resolve.openapi.json`: `frontend/app/docs/page.tsx`
- `/login`: `frontend/app/page.tsx`, `frontend/components/workspace-account.tsx`
- `/login?next=settings`: `frontend/components/merchant-login.tsx`, `frontend/components/merchant-sdk-keys.tsx`
- `/workspace`: `frontend/app/docs/layout.tsx`, `frontend/components/merchant-login.tsx`, `frontend/components/merchant-orders.tsx`, `frontend/components/operations-console.tsx`, `frontend/components/workspace-shell.tsx`
- `/workspace/operations`: `frontend/components/operations-console.tsx`, `frontend/components/workspace.tsx`
- `/workspace/orders`: `frontend/components/merchant-orders.tsx`, `frontend/components/operations-console.tsx`, `frontend/components/workspace-shell.tsx`, `frontend/components/workspace.tsx`

- `/workspace/settings`: `frontend/components/merchant-orders.tsx`, `frontend/components/operations-console.tsx`, `frontend/components/workspace-shell.tsx`, `frontend/components/workspace.tsx`
- `/workspace/settings#sdk-keys`: `frontend/components/merchant-orders.tsx`
- `http://127.0.0.1:8000/docs`: `frontend/components/ui.tsx`

Forms and source-level input constraints

This exhaustive input-control inventory records literal attributes; conditional controls and service-level cross-field rules are defined in the linked source and OpenAPI schemas. HTML validation is usability support; backend validation remains authoritative.

Component	Controls / validation attributes
`assurance.tsx`	name="enabled" type="checkbox"; type="number" name="days" min={1} max={365} required; name="fee" required; name="shipping"; type="checkbox" name="inspect"; type="checkbox" name="photos"; name="excluded"; name="instructions" required maxLength={3000}; name="instructions" required maxLength={3000}; name="address" maxLength={1000}; name="shipping"; name="label" type="file" accept="application/pdf" required; name="finding"; name="quantity" type="checkbox"; type="checkbox" name="evidence"; type="checkbox" name="override"; name="note" required maxLength={2000}; name="provider"; name="reference" required maxLength={100}; name="state"; name="note" required maxLength={2000}; name="refund" required maxLength={80}; name="type"; name="reference" required maxLength={100}; name="stage"; name="note" required maxLength={1000}; name="requirement"; name="evidence" type="checkbox"; name="note" required maxLength={2000}
`case-operations.tsx`	type="checkbox" name="promise"; type="datetime-local" name="expiry" required; name="dispute" required pattern="[A-Za-z0-9_-]+" maxLength={80}; name="reason"
`conversation.tsx`	maxLength={2000}
`customer-account.tsx`	name="email" type="email" maxLength={254} required
`customer-actions.tsx`	name="issue" required; name="description" maxLength={2000} required placeholder="Tell the merchant what happened. You can attach evidence after submitting."; name="evidence" type="file" accept=".pdf,.png,.jpg,.jpeg" required
`docs-navigation.tsx`	type="search" maxLength={100} placeholder="Find a guide…";
`forms.tsx`	name="title" placeholder="e.g. Refund for a returned item" required maxLength={160}; name="order" placeholder="Order reference or PayPal order ID" maxLength={mode === "new" ? 100 : 160} required; name="payment_mode"; name="legacy_id"; name="legacy_manual" placeholder="Resolve case ID"; name="capture" required maxLength={100}; name="captured" placeholder="100.00" pattern="[0-9]+(\.[0-9]{1,2})?" required; name="promise_amount" placeholder="30.00" pattern="[0-9]+(\.[0-9]{1,2})?"; name="agreement" placeholder="Required if you entered an agreed refund" maxLength={3000}; ; name="promise" required maxLength={3000}; name={spec.key}; name={spec.key} placeholder={spec.hint} required pattern={spec.money ? "[0-9]+(\\.[0-9]{1,2} maxLength={160}; name="source_note" required maxLength={3000} placeholder="Quote or describe the recorded agreement."; name="reference" maxLength={160} placeholder="Support ticket or receipt ID"; type="datetime-local" name="due_at"; type="checkbox" name="requires"; name="finding"; name="note" required maxLength={3000}; name="reference" maxLength={160} placeholder="Warehouse or support reference"; name="refund_amount"; name="note" required maxLength={5000} placeholder="Why is this refund approved?"; name="note" required maxLength={1000}
`helpdesk-panel.tsx`	name="ticket" pattern="[1-9][0-9]{0,15}" required
`merchant-customer.tsx`	; ; type="checkbox"

Component	Controls / validation attributes
merchant-login.tsx	name="key" type={visible ? "text" : "password"} placeholder="rslv_..." maxLength={125} required
merchant-orders.tsx	placeholder="Order or PayPal ID" maxLength={160}; placeholder="All apps" maxLength={64}
merchant-sdk-keys.tsx	aria-label="SDK server environment"
merchant-triage.tsx	; name="order" required maxLength={160} placeholder="e.g. ORD-4821"; name="email" type="email" maxLength={254} placeholder="buyer@example.com"; type="checkbox" name="send_email"; name="payment"; name="manual" placeholder="Resolve payment case ID"; type="checkbox" required; name="assignee" maxLength={80} placeholder="Name or team"; name="due_at" type="datetime-local"; name="waiting" type="checkbox"
operations-console.tsx	name={key} required maxLength={160}; name={key} type="password" maxLength={4096} placeholder={current ? "Leave blank to keep the saved value" : "Enter privately"}; type="checkbox" name="enabled"; type="checkbox" name="refunds_enabled"; type="checkbox" name="partial_refunds_enabled"; name="ticket" type="number" min={1} required; name="case" required; name="email" type="email" required; name="role"; aria-label="Private staff invitation"; name="role"; name="active" type="checkbox"; name="source" required pattern="[A-Za-z0-9][A-Za-z0-9_.-]{0,63}" maxLength={64}; type="password" aria-label="New SDK key"
return-workflow.tsx	; aria-label={`Return quantity for ${i.name ?? i.product_id} type="number" name={i.product_id} min={0} max={i.available_quantity}; name="reason" maxLength={2000} required; ; name="carrier" required maxLength={60}; name="tracking_code" required maxLength={100}; name="reference" required maxLength={160}; name="promise_id"; name="note" required maxLength={2000}
workspace-account.tsx	name="workspace" maxLength={100} required; name="name" maxLength={100} required; type="email" name="email" maxLength={254} required; aria-label="Password" type="password" name="password" minLength={12} maxLength={256} required
workspace.tsx	aria-label="Search cases" placeholder="Search cases..." maxLength={160}

Authentication and roles

- Staff email/password: signup, login, invitation acceptance, legacy owner claim. Passwords require 12-256 characters and use scrypt; DB throttles authentication attempts. No forgot-password/reset UI or endpoint exists.
- Merchant access key -> 8-hour database-backed browser session. Optional PayPal identity must already be linked. HttpOnly SameSite=Lax cookie; HTTPS sets Secure. Logout/revocation ends access. Browser mutation gateways enforce exact Origin.
- Buyers use expiring/revocable case capabilities, isolated customer sessions from email/Google when configured, and order-bound intake links. Buyer links do not grant merchant access.
- Owner: all merchant actions; manager: operational writes and refund approvals, no refund submission or secret/team administration; finance: refund approval/submission, dispute state/tracing; agent: ordinary case work; viewer: reads only. Owners/managers review dispute packages. Owner alone changes return policy/connections/keys/team. Backend enforcement is authoritative; legacy full merchant keys have owner authority.
- SDK commerce keys are restricted; support grants bind a key to one app source. The integrating application must establish the buyer's order ownership. No public end-customer registration exists.

Integrations

PayPal Sandbox OAuth/capture/order/refund/history/webhook verification; separate optional PayPal merchant identity; Google OIDC for buyers; Resend or SMTP/Nodemailer notices; Zendesk imports/webhooks; Shopify Admin GraphQL and EasyPost evidence reads; advisory Groq/OpenRouter inference. Optional features default off except workspace signup. No Stripe, bank receipt verification, live PayPal refund execution, label purchasing or automatic dispute submission exists.

API endpoint inventory

Each operation below is sourced from the canonical FastAPI-generated contract. Security denotes declared auth; public capability endpoints still validate tokens/signatures in application code.

Method	Path	Operation	Declared security
GET	`/health`	Health	Token/signature or public (see source)
POST	`/health`	Health	Token/signature or public (see source)
GET	`/ready`	Readiness	Token/signature or public (see source)
POST	`/cases`	Create Case	MerchantBearer
GET	`/cases`	List Cases	MerchantBearer
GET	`/cases/{case_id}`	Get Case	MerchantBearer
PUT	`/cases/{case_id}/commitment`	Update Commitment	MerchantBearer
GET	`/cases/{case_id}/timeline`	Get Timeline	MerchantBearer
POST	`/cases/{case_id}/refunds`	Record Refund	MerchantBearer
GET	`/cases/{case_id}/refunds`	List Refunds	MerchantBearer
PATCH	`/cases/{case_id}/refunds/{refund_id}`	Update Refund Status	MerchantBearer
POST	`/cases/{case_id}/condition-checks`	Record Condition Check	MerchantBearer
GET	`/cases/{case_id}/condition-checks`	List Condition Checks	MerchantBearer
GET	`/cases/{case_id}/reconciliation`	Get Reconciliation	MerchantBearer
GET	`/auth/me`	Get Identity	MerchantBearer
POST	`/cases/{case_id}/paypal-capture-verifications`	Verify Paypal Capture	MerchantBearer
GET	`/cases/{case_id}/paypal-capture-verifications`	List Paypal Capture Verifications	MerchantBearer
POST	`/cases/{case_id}/refund-approvals`	Create Refund Approval	MerchantBearer
GET	`/cases/{case_id}/refund-approvals`	List Refund Approvals	MerchantBearer

Method	Path	Operation	Declared security
GET	`/cases/{case_id}/refund-approvals/{approval_id}`	Get Refund Approval	MerchantBearer
POST	`/cases/{case_id}/refund-approvals/{approval_id}/revoke`	Revoke Refund Approval	MerchantBearer
GET	`/cases/{case_id}/refund-approvals/{approval_id}/events`	Get Refund Approval Events	MerchantBearer
POST	`/cases/{case_id}/refund-executions`	Execute Refund	MerchantBearer
GET	`/cases/{case_id}/refund-executions`	List Refund Executions	MerchantBearer
GET	`/cases/{case_id}/refund-executions/{execution_id}`	Get Refund Execution	MerchantBearer
POST	`/cases/{case_id}/refund-executions/{execution_id}/retry`	Retry Refund Execution	MerchantBearer
POST	`/cases/{case_id}/refund-executions/{execution_id}/refresh`	Refresh Refund Execution	MerchantBearer
GET	`/cases/{case_id}/refund-executions/{execution_id}/events`	Get Execution Events	MerchantBearer
POST	`/cases/{case_id}/paypal-refund-verifications`	Verify Paypal Refund	MerchantBearer
GET	`/cases/{case_id}/paypal-refund-verifications`	List Paypal Refund Verifications	MerchantBearer
POST	`/webhooks/paypal/sandbox`	Receive Paypal Webhook	Token/signature or public (see source)
POST	`/cases/{case_id}/paypal-refund-history/refresh`	Refresh Paypal History	MerchantBearer
POST	`/cases/{case_id}/close`	Close Case	MerchantBearer
POST	`/cases/{case_id}/reopen`	Reopen Case	MerchantBearer
POST	`/cases/{case_id}/attachments`	Upload Attachment	MerchantBearer
GET	`/cases/{case_id}/attachments`	List Attachments	MerchantBearer
GET	`/cases/{case_id}/attachments/{attachment_id}`	Download Attachment	MerchantBearer
GET	`/review-queue`	Get Review Queue	MerchantBearer
POST	`/review-queue/{exception_id}/resolve`	Resolve Exception	MerchantBearer
GET	`/resolution-cases/inbox`	Merchant Inbox	MerchantBearer

Method	Path	Operation	Declared security
GET	`/resolution-cases/{case_id}/customer-link`	Customer Link Status	MerchantBearer
POST	`/resolution-cases/{case_id}/customer-link`	Issue Customer Link	MerchantBearer
DELETE	`/resolution-cases/{case_id}/customer-link`	Revoke Customer Link	MerchantBearer
GET	`/resolution-cases/{case_id}/helpdesk`	Read Helpdesk	MerchantBearer
PUT	`/resolution-cases/{case_id}/helpdesk`	Link Helpdesk	MerchantBearer
GET	`/resolution-cases/{case_id}/customer-messages`	Merchant Read Messages	MerchantBearer
POST	`/resolution-cases/{case_id}/customer-messages`	Merchant Send Message	MerchantBearer
POST	`/customer-portal/view`	Customer View	Token/signature or public (see source)
POST	`/customer-portal/messages`	Customer Send Message	Token/signature or public (see source)
POST	`/resolution-cases/intake-invites`	Create Intake Invite	MerchantBearer
GET	`/customer-account/capabilities`	Customer Account Capabilities	Token/signature or public (see source)
POST	`/customer-account/google/start`	Start Google Sign In	Token/signature or public (see source)
POST	`/customer-account/google/finish`	Finish Google Sign In	Token/signature or public (see source)
POST	`/customer-account/request-sign-in`	Request Customer Sign In	Token/signature or public (see source)
POST	`/customer-account/redeem`	Redeem Customer Sign In	Token/signature or public (see source)
GET	`/customer-account/cases`	List Customer Account Cases	Token/signature or public (see source)
POST	`/customer-account/cases/{case_id}/open`	Open Customer Account Case	Token/signature or public (see source)
POST	`/customer-account/sign-out`	Customer Sign Out	Token/signature or public (see source)
POST	`/customer-portal/intake-view`	Intake View	Token/signature or public (see source)
POST	`/customer-portal/intake`	Submit Intake	Token/signature or public (see source)
POST	`/customer-portal/evidence`	Customer Upload Evidence	Token/signature or public (see source)

Method	Path	Operation	Declared security
POST	`/customer-portal/evidence/{file_id}`	Buyer Download Evidence	Token/signature or public (see source)
GET	`/resolution-cases/{case_id}/customer-evidence`	Merchant Customer Evidence	MerchantBearer
GET	`/resolution-cases/{case_id}/customer-evidence/{file_id}`	Merchant Download Evidence	MerchantBearer
GET	`/resolution-cases/{case_id}/operations`	Case Operations	MerchantBearer
PUT	`/resolution-cases/{case_id}/operations`	Update Case Operations	MerchantBearer
POST	`/resolution-cases/{case_id}/operations/read`	Mark Customer Messages Read	MerchantBearer
GET	`/merchant-auth/capabilities`	Capabilities	Token/signature or public (see source)
POST	`/merchant-auth/session/key`	Key Login	Token/signature or public (see source)
GET	`/merchant-auth/session`	Merchant Session	MerchantBearer
DELETE	`/merchant-auth/session`	Logout	MerchantBearer
POST	`/merchant-auth/paypal/start`	Start Paypal	MerchantBearer
POST	`/merchant-auth/paypal/finish`	Finish Paypal	MerchantBearer
DELETE	`/merchant-auth/paypal/link`	Unlink Paypal	MerchantBearer
GET	`/merchant-auth/commerce-keys`	List Commerce Keys	MerchantBearer
POST	`/merchant-auth/commerce-keys`	Issue Commerce Key	MerchantBearer
DELETE	`/merchant-auth/commerce-keys/{key_id}`	Revoke Commerce Key	MerchantBearer
GET	`/resolution-cases/promise-health`	Promises Needing Attention	MerchantBearer
GET	`/resolution-cases/{case_id}/promise-health`	Case Promise Health	MerchantBearer
GET	`/resolution-cases/{case_id}/next-action`	Case Next Action	MerchantBearer
POST	`/resolution-cases`	Create Resolution	MerchantBearer
GET	`/resolution-cases`	List Resolutions	MerchantBearer
POST	`/resolution-cases/upsert`	Upsert Evidence Case	MerchantBearer
GET	`/resolution-cases/{case_id}`	Get Resolution	MerchantBearer

Method	Path	Operation	Declared security
POST	`/resolution-cases/{case_id}/evidence/records`	Link Evidence Record	MerchantBearer
POST	`/resolution-cases/{case_id}/evidence/events`	Submit Merchant Evidence	MerchantBearer
POST	`/resolution-cases/{case_id}/evidence/refresh`	Refresh Case Evidence	MerchantBearer
PUT	`/resolution-cases/{case_id}/payment-link`	Link Resolution Payment	MerchantBearer
POST	`/resolution-cases/{case_id}/promises`	Record Promise	MerchantBearer
PUT	`/resolution-cases/{case_id}/promises/{promise_id}`	Revise Promise	MerchantBearer
POST	`/resolution-cases/{case_id}/promises/{promise_id}/evidence`	Record Promise Evidence	MerchantBearer
POST	`/resolution-cases/{case_id}/promises/{promise_id}/refresh`	Refresh Promise	MerchantBearer
GET	`/resolution-cases/{case_id}/timeline`	Resolution Timeline	MerchantBearer
GET	`/resolution-cases/{case_id}/evidence-bundle`	Evidence Bundle	MerchantBearer
POST	`/commerce/orders/sync`	Sync Commerce Order	MerchantBearer
POST	`/commerce/orders/register`	Register Commerce Order	MerchantBearer
GET	`/commerce/orders`	List Commerce Orders	MerchantBearer
GET	`/commerce/orders/{order_id}`	Get Registered Order	MerchantBearer
POST	`/commerce/orders/{order_id}/case`	Open Commerce Case	MerchantBearer
GET	`/resolution-cases/{case_id}/commerce`	Get Commerce Order	MerchantBearer
POST	`/resolution-cases/{case_id}/commerce/refresh`	Refresh Commerce Order	MerchantBearer
GET	`/resolution-cases/{case_id}/commerce/contact`	Get Commerce Contact	MerchantBearer
GET	`/resolution-cases/{case_id}/investigations`	Investigations	MerchantBearer
POST	`/resolution-cases/{case_id}/investigations/{investigation_id}/review`	Review Investigation	MerchantBearer
POST	`/resolution-cases/{case_id}/ai/extract-promises`	Extract Promise Proposals	MerchantBearer

Method	Path	Operation	Declared security
GET	`/ai/usage`	Extraction Usage	MerchantBearer
POST	`/resolution-cases/{case_id}/ai/explain-next-action`	Explain Next Action	MerchantBearer
POST	`/platform/account/signup`	Signup	Token/signature or public (see source)
POST	`/platform/account/login`	Login	Token/signature or public (see source)
POST	`/platform/account/claim`	Claim Owner	MerchantBearer
POST	`/platform/account/accept-invite`	Accept Invite	Token/signature or public (see source)
GET	`/platform/me`	Me	MerchantBearer
GET	`/platform/team`	Team	MerchantBearer
POST	`/platform/team/invites`	Invite	MerchantBearer
PUT	`/platform/team/{staff_id}`	Update Staff	MerchantBearer
GET	`/platform/audit`	Audit History	MerchantBearer
GET	`/platform/sdk-keys`	Sdk Keys	MerchantBearer
POST	`/platform/sdk-keys`	Issue Sdk Key	MerchantBearer
POST	`/platform/support-keys`	Issue Support Key	MerchantBearer
DELETE	`/platform/sdk-keys/{key_id}`	Revoke Sdk Key	MerchantBearer
GET	`/platform/connections`	Connections	MerchantBearer
PUT	`/platform/connections/{provider}`	Save Connection	MerchantBearer
POST	`/platform/connections/{provider}/check`	Check Connection	MerchantBearer
GET	`/platform/imports`	Imports	MerchantBearer
POST	`/platform/imports/zendesk`	Import Ticket	MerchantBearer
GET	`/platform/imports/{import_id}`	Imported Ticket	MerchantBearer
PUT	`/platform/imports/{import_id}/match`	Match Ticket	MerchantBearer
GET	`/platform/dashboard`	Dashboard	MerchantBearer
GET	`/platform/watchdog`	Watchdog	MerchantBearer
POST	`/platform/watchdog/scan`	Scan	MerchantBearer
GET	`/platform/cases/{case_id}/offers`	Offers	MerchantBearer
POST	`/platform/cases/{case_id}/offers`	Publish Offer	MerchantBearer

Method	Path	Operation	Declared security
GET	`/platform/cases/{case_id}/support-requests`	Case Support Requests	MerchantBearer
GET	`/platform/cases/{case_id}/returns`	Returns	MerchantBearer
POST	`/platform/cases/{case_id}/returns`	Request Return	MerchantBearer
PUT	`/platform/cases/{case_id}/returns/{return_id}`	Progress Return	MerchantBearer
GET	`/platform/cases/{case_id}/dispute-packages`	Packages	MerchantBearer
POST	`/platform/cases/{case_id}/dispute-packages`	Build Package	MerchantBearer
GET	`/platform/cases/{case_id}/dispute-packages/{package_id}`	Read Package	MerchantBearer
POST	`/platform/cases/{case_id}/dispute-packages/{package_id}/review`	Review Package	MerchantBearer
GET	`/platform/cases/{case_id}/dispute-packages/{package_id}/export`	Export Package	MerchantBearer
POST	`/platform/buyer/offers`	Buyer Offers	Token/signature or public (see source)
POST	`/platform/buyer/decision`	Buyer Decision	Token/signature or public (see source)
POST	`/platform/buyer/returns`	Buyer Returns	Token/signature or public (see source)
POST	`/platform/buyer/returns/request`	Buyer Request Return	Token/signature or public (see source)
POST	`/aftercare/issues`	Open Aftercare Issue	MerchantBearer
POST	`/aftercare/status`	Aftercare Status	MerchantBearer
POST	`/aftercare/portal`	Aftercare Portal	MerchantBearer
GET	`/platform/return-policy`	Policy	MerchantBearer
PUT	`/platform/return-policy`	Policy Save	MerchantBearer
POST	`/cases/{case_id}/disputes`	Dispute Record	MerchantBearer
GET	`/cases/{case_id}/disputes`	Dispute List	MerchantBearer
GET	`/platform/cases/{case_id}/disputes`	Resolution Disputes	MerchantBearer
POST	`/platform/cases/{case_id}/disputes`	Resolution Dispute Record	MerchantBearer
POST	`/platform/cases/{case_id}/refund-traces`	Trace Record	MerchantBearer

Method	Path	Operation	Declared security
POST	`/platform/cases/{case_id}/dispute-documents`	Dispute Document	MerchantBearer
GET	`/platform/cases/{case_id}/returns/{return_id}/guide`	Return Guide	MerchantBearer
PUT	`/platform/cases/{case_id}/returns/{return_id}/guide`	Save Return Guide	MerchantBearer
POST	`/platform/cases/{case_id}/returns/{return_id}/inspection`	Inspect Return	MerchantBearer
POST	`/platform/cases/{case_id}/returns/{return_id}/labels`	Upload Label	MerchantBearer
POST	`/aftercare/journey`	Refund Journey	MerchantBearer
POST	`/aftercare/context`	Support Context	MerchantBearer
POST	`/aftercare/received`	Received Refund	MerchantBearer
POST	`/aftercare/issues/lookup`	Issue Lookup	MerchantBearer
POST	`/aftercare/events`	Event Pull	MerchantBearer
POST	`/aftercare/returns/eligibility`	Check Eligibility	MerchantBearer
POST	`/aftercare/returns`	Support Returns	MerchantBearer
POST	`/aftercare/returns/request`	Support Return Request	MerchantBearer
POST	`/aftercare/returns/guide`	Support Return Guide	MerchantBearer
POST	`/aftercare/returns/label`	Support Label	MerchantBearer
POST	`/platform/buyer/assurance`	Buyer Assurance	Token/signature or public (see source)
POST	`/platform/buyer/returns/guide`	Buyer Guide	Token/signature or public (see source)
POST	`/platform/buyer/returns/label`	Buyer Label	Token/signature or public (see source)
POST	`/platform/buyer/return-policy`	Buyer Return Policy	Token/signature or public (see source)
POST	`/webhooks/zendesk/{merchant_id}`	Zendesk Hook	Token/signature or public (see source)

Total: **175 operations** across **150 API paths**.

Frontend API gateways and webhook routes

Next.js entry points constrain downstream paths, credentials, origins and body sizes. Server-only service addresses/secrets are not client configuration.

Route	Exported methods	Source
/api/auth/paypal/[action]	POST, GET, DELETE	frontend/app/api/auth/paypal/[action]/route.ts
/api/customer-account/[...path]	GET, POST	frontend/app/api/customer-account/[...path]/route.ts
/api/customer-portal/[...path]	POST	frontend/app/api/customer-portal/[...path]/route.ts
/api/platform/[...path]	GET, POST, PUT, DELETE	frontend/app/api/platform/[...path]/route.ts
/api/platform-account/[action]	POST	frontend/app/api/platform-account/[action]/route.ts
/api/platform-buyer/[...path]	POST	frontend/app/api/platform-buyer/[...path]/route.ts
/api/resolve/[...path]	GET, POST, PUT, DELETE	frontend/app/api/resolve/[...path]/route.ts
/api/sdk/[...path]	GET, POST	frontend/app/api/sdk/[...path]/route.ts
/api/session	GET, POST, DELETE	frontend/app/api/session/route.ts
/webhooks/zendesk/[merchant]	POST	frontend/app/webhooks/zendesk/[merchant]/route.ts

Database operation inventory

Database operations reside in services, not only route handlers. Tables and mutations below are extracted statically from backend Python/SQL; dynamically built queries must also be reviewed in their source.

Table	Observed SQL operations
aftercare_apps	CREATE TABLE, INSERT INTO, SELECT
aftercare_issues	CREATE TABLE, INSERT INTO, SELECT
assurance_dispute_events	CREATE TABLE, INSERT INTO, SELECT
assurance_disputes	CREATE TABLE, INSERT INTO, SELECT
assurance_events	CREATE TABLE, INSERT INTO, SELECT
assurance_policies	CREATE TABLE, INSERT INTO, SELECT
assurance_projections	CREATE TABLE, INSERT INTO, SELECT, UPDATE
assurance_receipts	CREATE TABLE, INSERT INTO, SELECT
assurance_refund_traces	CREATE TABLE, INSERT INTO, SELECT
assurance_return_guides	CREATE TABLE, INSERT INTO, SELECT
assurance_return_labels	CREATE TABLE, INSERT INTO, SELECT
assurance_return_reviews	CREATE TABLE, INSERT INTO, SELECT
captures	CREATE TABLE, INSERT INTO, SELECT, UPDATE

Table	Observed SQL operations
case_attachments	CREATE TABLE, INSERT INTO, SELECT
case_events	CREATE TABLE, INSERT INTO, SELECT
cases	CREATE TABLE, INSERT INTO, SELECT, UPDATE
commerce_order_snapshots	CREATE TABLE, CREATE TABLE IF NOT EXISTS, SELECT
commerce_orders	CREATE TABLE, INSERT INTO, SELECT, UPDATE
condition_checks	CREATE TABLE, INSERT INTO, SELECT
customer_account_links	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT
customer_case_contacts	CREATE TABLE, INSERT INTO, SELECT
customer_evidence	CREATE TABLE, INSERT INTO, SELECT
customer_google_flows	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT
customer_intake_invites	CREATE TABLE, INSERT INTO, SELECT, UPDATE
customer_mail_outbox	CREATE TABLE, INSERT INTO, SELECT, UPDATE
customer_portal_links	CREATE TABLE, INSERT INTO, SELECT, UPDATE
customer_portal_messages	CREATE TABLE, INSERT INTO, SELECT
customer_sessions	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT
customer_signins	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT, UPDATE
information_schema	SELECT
investigation_audit	CREATE TABLE, INSERT INTO
investigation_jobs	CREATE TABLE, INSERT INTO, SELECT, UPDATE
investigation_reviews	CREATE TABLE, INSERT INTO, SELECT
investigations	CREATE TABLE, INSERT INTO, SELECT
legacy_captures	SELECT
legacy_cases	SELECT
legacy_condition_checks	SELECT
legacy_refunds	SELECT
merchant_api_keys	CREATE TABLE, INSERT INTO, SELECT, UPDATE
merchant_paypal_flows	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT, UPDATE
merchant_paypal_links	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT
merchant_web_sessions	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT
merchants	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT
paypal_capture_verifications	CREATE TABLE, INSERT INTO, SELECT
paypal_refund_history	CREATE TABLE, INSERT INTO, SELECT
paypal_refund_verifications	CREATE TABLE, INSERT INTO, SELECT

Table	Observed SQL operations
paypal_refund_verifications_v7	SELECT
paypal_webhook_events	CREATE TABLE, INSERT INTO, SELECT, UPDATE
pg_constraint	SELECT
pg_trigger	SELECT
platform_alerts	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_audit	CREATE TABLE, INSERT INTO, SELECT
platform_auth_limits	CREATE TABLE, INSERT INTO, SELECT
platform_connections	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_imports	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_invites	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_offers	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_packages	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_returns	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_sessions	CREATE TABLE, DELETE FROM, INSERT INTO, SELECT
platform_staff	CREATE TABLE, INSERT INTO, SELECT, UPDATE
platform_ticket_messages	CREATE TABLE, INSERT INTO, SELECT
platform_webhook_jobs	CREATE TABLE, INSERT INTO, SELECT, UPDATE
pragma_table_info	SELECT
provider_refresh_schedule	CREATE TABLE, INSERT INTO, UPDATE
refund_approval_events	CREATE TABLE, INSERT INTO, SELECT
refund_approvals	CREATE TABLE, INSERT INTO, SELECT, UPDATE
refund_execution_events	CREATE TABLE, INSERT INTO, SELECT
refund_execution_events_v7	SELECT
refund_executions	CREATE TABLE, INSERT INTO, SELECT, UPDATE
refund_executions_v7	SELECT
refunds	CREATE TABLE, INSERT INTO, SELECT, UPDATE
resolution_cases	CREATE TABLE, INSERT INTO, SELECT, UPDATE
resolution_events	CREATE TABLE, INSERT INTO, SELECT
resolution_evidence_events	CREATE TABLE, INSERT INTO, SELECT
resolution_evidence_links	CREATE TABLE, INSERT INTO, SELECT
resolution_evidence_refs	CREATE TABLE, INSERT INTO, SELECT
resolution_evidence_state	CREATE TABLE, INSERT INTO, SELECT
resolution_helpdesk	CREATE TABLE, INSERT INTO, SELECT

Table	Observed SQL operations
resolution_operations	CREATE TABLE, INSERT INTO, SELECT, UPDATE
resolution_promises	CREATE TABLE, INSERT INTO, SELECT, UPDATE
review_exceptions	CREATE TABLE, INSERT INTO, SELECT, UPDATE
schema_migrations	CREATE TABLE IF NOT EXISTS, INSERT INTO, SELECT
sqlite_master	SELECT
sqlite_sequence	SELECT

Environment variables (names only)

Never copy configured values into documentation. Ordinary API settings use backend dotenv/process environment; inference settings use process environment only. Frontend gateways read only server variables. Provider-specific required combinations are documented in `.env.example` and the existing integration guides.

- `RESOLVE_DATABASE_PATH` (Path)
- `RESOLVE_DATABASE_URL` (SecretStr | None)
- `RESOLVE_MERCHANT_WEB_ORIGIN` (str | None)
- `RESOLVE_PAYPAL_LOGIN_ENABLED` (bool)
- `RESOLVE_PAYPAL_LOGIN_ENVIRONMENT` (Literal['sandbox', 'live'])
- `RESOLVE_PAYPAL_LOGIN_CLIENT_ID` (SecretStr | None)
- `RESOLVE_PAYPAL_LOGIN_CLIENT_SECRET` (SecretStr | None)
- `RESOLVE_PAYPAL_SANDBOX_ENABLED` (bool)
- `RESOLVE_PAYPAL_SANDBOX_REFUNDS_ENABLED` (bool)
- `RESOLVE_PAYPAL_SANDBOX_PARTIAL_REFUNDS_ENABLED` (bool)
- `RESOLVE_PAYPAL_SANDBOX_WEBHOOK_ID` (str | None)
- `RESOLVE_PAYPAL_RESOLVE_MERCHANT_ID` (UUID | None)
- `RESOLVE_PAYPAL_SANDBOX_CLIENT_ID` (SecretStr | None)
- `RESOLVE_PAYPAL_SANDBOX_CLIENT_SECRET` (SecretStr | None)
- `RESOLVE_PAYPAL_SANDBOX_MERCHANT_ID` (str | None)
- `RESOLVE_RESOLUTION_CONNECTOR_MERCHANT_ID` (UUID | None)
- `RESOLVE_SHOPIFY_SHOP_DOMAIN` (str | None)
- `RESOLVE_SHOPIFY_ADMIN_TOKEN` (SecretStr | None)
- `RESOLVE_EASYPOST_API_KEY` (SecretStr | None)
- `RESOLVE_AUTO_PROVIDER_REFRESH_ENABLED` (bool)
- `RESOLVE_PROMISE_OVERDUE_DAYS` (int)
- `RESOLVE_CUSTOMER_EMAIL_ENABLED` (bool)

- `RESOLVE_CUSTOMER_EMAIL_TRANSPORT` (Literal['resend', 'smtp'])
- `RESOLVE_CUSTOMER_EMAIL_MERCHANT_ID` (UUID | None)
- `RESOLVE_CUSTOMER_PUBLIC_ORIGIN` (str | None)
- `RESOLVE_CUSTOMER_SIGNING_KEY` (SecretStr | None)
- `RESOLVE_RESEND_API_KEY` (SecretStr | None)
- `RESOLVE_RESEND_FROM` (str | None)
- `RESOLVE_SMTP_HOST` (str | None)
- `RESOLVE_SMTP_PORT` (int)
- `RESOLVE_SMTP_SECURE` (bool)
- `RESOLVE_SMTP_USER` (str | None)
- `RESOLVE_SMTP_PASSWORD` (SecretStr | None)
- `RESOLVE_SMTP_FROM` (str | None)
- `RESOLVE_CUSTOMER_GOOGLE_ENABLED` (bool)
- `RESOLVE_GOOGLE_CLIENT_ID` (SecretStr | None)
- `RESOLVE_GOOGLE_CLIENT_SECRET` (SecretStr | None)
- `RESOLVE_HELPDESK_MERCHANT_ID` (UUID | None)
- `RESOLVE_ZENDESK_SUBDOMAIN` (str | None)
- `RESOLVE_ZENDESK_OAUTH_TOKEN` (SecretStr | None)
- `RESOLVE_PLATFORM_ENCRYPTION_KEY` (SecretStr | None)
- `RESOLVE_MERCHANT_SIGNUP_ENABLED` (bool)

Additional directly referenced process variables: `GROQ_DATA_POLICY`, `RESOLVE_AI_`, `RESOLVE_AI_DATA_POLICY`, `RESOLVE_AI_GROQ_KEY`, `RESOLVE_AI_TIMEOUT_SECONDS`, `RESOLVE_BACKEND_ALLOW_PRIVATE_HTTP`, `RESOLVE_BACKEND_URL`, `RESOLVE_CUSTOMER_PUBLIC_ORIGIN`, `RESOLVE_DATABASE_PATH`, `RESOLVE_DATABASE_URL`, `RESOLVE_EXTRACTION_`, `RESOLVE_ISOLATED_TEST`, `RESOLVE_MERCHANT_WEB_ORIGIN`, `RESOLVE_PLATFORM_ENCRYPTION_KEY`.

Startup and pre-existing deployment state

Backend: `cd backend; uv sync --locked; uv run uvicorn app.main:app --host 127.0.0.1 --port 8000`. Frontend: `npm --prefix frontend ci`, `npm --prefix frontend run build:preview`, then `npm --prefix frontend run preview`. Four workers: `app.worker`, `app.followup_worker`, `app.investigation_worker`, `app.operations_worker` (each supports `--loop`).

At audit start there was no Dockerfile, infrastructure manifest or CI workflow. Hosted setup existed only as instructions and `start:hosted`. The owner has an existing configured runtime; all audit tests must use isolated settings and disposable databases. Existing dirty files are preserved. The checkout-store example is a separate sample with its own README, database and deployment lifecycle.

Separate checkout-store example

`examples/checkout-store` is a standalone Node 24 HTTP server with built-in SQLite, six-product catalog, cart, Sandbox redirect/capture, receipt and embedded order support. It is not a hidden Resolve dashboard and is not deployed by the main Render Blueprint. Its browser page is `/`; catalog/cart/receipt/support are states within that page. Static assets are local CSS/JS/SVG. The browser owns receipts through an HttpOnly session; another browser gets 404. Server-side quoting rejects client-supplied prices and capture retries reuse identity. Shipping is not implemented.

Its API routes are `GET /api/catalog`, `POST /api/orders`, `GET /api/orders/{id}`, and `POST /api/orders/{id}/{action}` for `capture`, `verify`, `support`, `resolution`, `support-link`, `refund-journey`, `context`, and `return-eligibility`. Store-specific `SHOP_*` settings and Resolve integration values are documented in its README/source. These are additional to the 175 FastAPI operations above. The PayPal approval page used in the test is a mock redirect, not an application route or actual PayPal account consent.

Practical merchant and buyer guide

Source: docs/RESOLVE_GUIDE.md

Resolve guide

SDK 0.7 · PayPal Sandbox

A practical guide for merchants, buyers and developers. All examples use placeholders or synthetic order data.

Contents

• Start your first resolution (#start-your-first-resolution)

• Run the merchant workspace (#run-the-merchant-workspace)

• Help a buyer without starting over (#help-a-buyer-without-starting-over)

• Guide returns and replacements (#guide-returns-and-replacements)

• Review refunds and disputes (#review-refunds-and-disputes)

• Integrate the SDK (#integrate-the-sdk)

• Use the API reliably (#use-the-api-reliably)

• Connect supported services (#connect-supported-services)

• Run and configure Resolve (#run-and-configure-resolve)

• Troubleshoot common problems (#troubleshoot-common-problems)

Start your first resolution

Learn the basics and try a merchant-to-buyer case without connecting a payment account.

What Resolve does

Resolve is a payment aftercare platform. It brings a customer issue, merchant commitments, evidence, messages and resolution progress together. Merchants work in a workspace; buyers use a private case link or the support flow inside a connected store.

A request is what a customer asks for. A promise is what the merchant agrees to do. Evidence records what happened. A refund approval authorizes one reviewed action; submission and provider confirmation are separate steps.

> **Current scope**
>
> Payment verification and refund execution use PayPal Sandbox. SDK 0.7 is a local Node.js preview. Physical returns, tracing references and manually entered dispute updates retain their merchant-recorded source.

Create a practice case

• Create a workspace or sign in (/account)

1. Open /account and choose Create workspace. Enter your store name, name, email and a password of at least 12 characters. If you registered already, sign in.

2. Open Cases -> New case. Enter "Customer received a damaged item" and optional order reference TEST-001.

3. Choose No linked payment, then Create case. This practice case needs no provider credentials.

4. Choose Add promise. Record the agreed remedy, its source, due date and any prerequisites.

5. Open Customer portal -> Create customer link. Open the issued link in a private browser window to try the buyer view.

6. Send a message or upload sample evidence. Refresh the merchant conversation to see the update.

Already using a full merchant key? Sign in with that key at /login, then open /account?claim=1 to create the first individual owner for the same workspace. Creating a separate workspace gives it separate records.

Choose your next step

A standalone case supports promises, messages, evidence and sharing. Item returns require a registered order with items. Refund reconciliation requires a linked payment and the relevant provider configuration.

• Merchant workspace (#run-the-merchant-workspace)

• Buyer portal (#help-a-buyer-without-starting-over)

• SDK integration (#integrate-the-sdk)

• Run Resolve locally (#run-and-configure-resolve)

Run the merchant workspace

Handle cases, evidence, offers, overdue work and individual staff permissions.

A practical daily workflow

1. Open Cases. Search by order or case reference and use workflow filters or At risk to find work needing attention.

2. Select a case. Read the next step, money summary, promises, due dates and evidence sources.

3. Record only the remedy actually agreed to, its source and prerequisites. Customer preferences remain separate until a merchant records a promise.

4. Use Customer portal to send a public update, collect evidence and create or revoke a private buyer link.

5. Open Operations -> Missed promises -> Scan workspace for overdue work and customer-update drafts. Review and send drafts through your chosen channel.

6. Use Manage case to close completed work. Case closure and verified payment completion remain separate facts.

Read evidence by source

Label	Meaning
Merchant recorded	A staff member or connected app recorded a fact or finding.
Provider confirmed	A supported provider returned evidence for this record; check timestamp and scope.
Needs review	Evidence is missing, conflicting, stale or awaiting a required decision.
Pending	An operation or prerequisite has not reached its final outcome.

The timeline loads when opened. Secondary Zendesk, investigation, offer, return and dispute sections load on expansion. Refresh explicitly when you need newly arrived case data. AI suggestions and investigation reviews are advisory; they never authorize payment.

Let a buyer accept an offer

1. Record current promise terms in the case.

2. Expand Buyer resolution offers. Choose up to four remedies and an expiry, then publish.

3. Share the current private buyer link. The buyer accepts one exact version or declines.

4. If terms change, publish current terms. Expired, replaced or stale offers cannot be accepted. Refunds still need separate approval and payment checks.

People and permissions

Role	Typical access
Owner	Workspace, team, connections, policies and financial work.
Manager	Case work and refund approval; no execution.
Finance	Refund approval and execution; owner-only configuration remains restricted.
Agent	Case handling, messages and ordinary evidence work; no financial approval or execution.
Viewer	Read-only access.

Invite staff under Operations -> Team and share the private invitation link. It expires after two days and is single-use; it is not emailed automatically. Role changes or disabling a member revoke their sessions. Policy exceptions and sensitive actions have additional role checks.

Help a buyer without starting over

Use a private case to exchange messages, review offers, save labels and understand refund progress.

Open the right case

Use the private link supplied by the merchant, or open support from your authenticated store order page. Keep the case link private. A replacement link invalidates the earlier case link. My cases can show cases tied to a verified email when optional sign-in is configured.

> **Customer ownership**
>
> A merchant app must authenticate its customer and verify order ownership before calling the SDK. Knowing an order number alone is not a sign-in method.

Work through your resolution

1. Read the agreed remedy and next action. A requested refund or replacement may still need merchant agreement.

2. Reply in the existing conversation. Use Refresh messages to check for new replies.

3. Attach relevant receipts or photos. Case uploads support PDF, PNG and JPEG subject to the displayed size limit.

4. Review a current offer and accept the option you agree to. Remaining conditions still apply.

5. If returns are available, select actual order items and quantities, add a reason and request review.

6. Wait for approval and instructions. Save the uploaded PDF label so you can print it later, and keep your drop-off receipt.

Understand refund progress

Status or reference	What it establishes
Promised	An agreed commitment; no confirmed transfer yet.
Provider pending	The provider operation is still processing.
Sent to original payment method	Processor completion, without independent bank-receipt verification.
Needs verification or review	Evidence is unavailable, stale or conflicting. Unknown amounts are not zero.
ARN / STAN / RRN / bank reference	A merchant-supplied reference to help trace a recorded refund.

If a store uses SDK receipt acknowledgement, it is buyer-reported and changes no payment ledger. Private merchant tracing notes are not shown. Ask for a new case link if yours expired or was revoked. Revoking a server SDK key does not itself revoke previously issued buyer links.

• Returns and replacements (#guide-returns-and-replacements)

• Troubleshooting (#troubleshoot-common-problems)

Guide returns and replacements

Review real order items, disclose terms, save labels and record fulfillment milestones.

Before requesting an item return

Register the order and item quantities through the SDK or commerce API and associate it with the case. A typed order reference does not create an item catalog. Existing non-rejected requests reserve quantities and reduce what remains available.

• Register orders with the SDK (#integrate-the-sdk)

Configure review rules

1. As an owner, open Operations -> Connections -> Return policy and review checks.

2. Enable the policy and set the delivery-based return window, shipping payer, disclosed fee, evidence requirements and inspection requirement.

3. Add products needing an exception and write default instructions, then save the current revision.

4. Review flagged dates, exclusions, missing evidence or repeated requests. Owners/managers can record documented policy exceptions where permitted.

> **Costs and eligibility**
>
> An inactive policy does not quote a free return. Confirm eligibility, shipping costs and fees before shipping. Flags are review prompts, not proof of fraud. No fee is charged automatically and quantity limits cannot be overridden.

Give clear instructions and a reusable label

1. Open Guided returns and replacements in the case. Review the request and approve or decline it.

2. Expand Return instructions, saved labels and inspection. Save the address, instructions and shipping payer.

3. Upload an already arranged PDF return label, up to 2 MiB. Up to ten labels are stored per return; identical bytes reuse the existing label.

4. The buyer opens the same section in the private portal and chooses Save label PDF. They can print it later.

Resolve does not purchase postage or arrange carrier pickup. Actual shipment stays with the merchant and carrier.

Keep milestones separate

Stage	Record it when
Requested / approved	The buyer asks and the merchant reviews.
Return in transit	The merchant records the carrier, tracking and shipment reference.
Warehouse received	The warehouse confirms receipt; carrier delivery is a separate fact.
Inspection passed	A current item/quantity review satisfies active policy and return revisions.
Replacement shipped / complete	A replacement shipment or final workflow result is recorded.

These are merchant-recorded findings. A dependent refund still needs its acceptance, condition, evidence and payment gates. Changed policy or return terms can make an earlier review stale.

Review refunds and disputes

Understand balances, separate approvals, overlap holds and reason-specific evidence packages.

Read the money summary

Link the correct payment record. Refund promises are cumulative commitments, not instructions to add the whole amount again on each attempt. Cases sharing a capture use one shared balance.

Amount	How to read it
Promised	The current agreed cumulative amount.
Completed	Completed ledger observations; current independent provider evidence is shown separately.
Pending	Refunds still processing, kept separate from completed amounts.
Outstanding	What remains after applicable completed and pending amounts.

The Operations dashboard labels recorded balances separately from provider proof. Unavailable provider history is not a verified zero.

Follow the financial review flow

1. Check the linked capture, current PayPal history, prerequisites, conditions and any required buyer acceptance.

2. Refresh evidence where necessary. Missing/stale proof, conflicting history, disputes or uncertain previous submissions can block another action.

3. An owner, manager or finance user reviews an exact permitted amount and records a state-bound approval.

4. An owner or finance user submits the separately approved Sandbox refund when writes are explicitly enabled. Approval alone sends nothing.

5. Check execution and provider outcome. Reconcile an uncertain response using its original identity rather than issuing another refund.

> **Payment scope**
>
> This write path is PayPal Sandbox with explicit opt-ins. A support key, buyer acceptance or AI suggestion cannot execute a refund.

Trace a recorded refund

An owner/finance user can add a provider-supplied ARN, STAN, RRN or bank reference for a refund already recorded on this capture. The buyer sees the labelled reference and stage; the private note stays private. Processor completion does not prove bank receipt, and Resolve does not fetch these references from a live bank.

Hold overlapping remedies and prepare evidence

After linking the payment, an owner/manager/finance user can record reviewed dispute updates. API integrations retain stable event IDs and increasing versions. Older updates are ignored; conflicts are rejected. Active or reimbursed disputes hold the shared capture. A later manual close cannot erase reimbursement or override current provider-confirmed active dispute evidence.

Stripe and bank names here describe merchant-recorded references, not live connectors. A new hold cannot recall an already-sent payment request.

1. Classify owned delivery, product, policy, authorization, cancellation or credit evidence.

2. Open Dispute evidence packages and choose the actual reason. Prepare the immutable snapshot.

3. Inspect source labels and missing requirements. Replacement delivery is not original-delivery evidence.

4. An owner/manager reviews before export. Changed sources require a fresh package and review.

A checklist shows recorded support, not authenticity or a guaranteed outcome. Exporting does not submit anything to PayPal.

Integrate the SDK

Put order support inside your store with scoped server keys and durable request IDs.

Install the local preview

SDK 0.7 requires server-side Node.js 22.12 or newer. It is not published to npm. Pack the repository SDK, then install the archive in your merchant app.

```
# From the Resolve repository
New-Item -ItemType Directory -Force .local/artifacts
Set-Location sdk/typescript
npm pack --pack-destination ../../.local/artifacts

# From your merchant app; replace the absolute path
npm install C:/path/to/resolve-paypal-sdk-0.7.0.tgz
```

Create a support integration key

1. As an owner, open Operations -> Connections -> Customer support inside your store.

2. Choose a stable source such as my-store. Generate a support key and save the copied values on your app server.

3. Use the same source for order registration and support. Order-only keys do not automatically gain aftercare access.

```
RESOLVE_API_URL=http://127.0.0.1:3000/api/sdk
RESOLVE_API_KEY=<your-support-integration-key>
RESOLVE_APP_SOURCE=my-store
```

The website gateway exposes allowed commerce/aftercare actions. Full merchant helpers require direct FastAPI and full-key permissions. Never put a key in browser code or NEXT_PUBLIC variables.

Register trusted order data

```
import { createResolve } from "@resolve-paypal/sdk";
const resolve = createResolve();

// Read this from your app's trusted order record.
await resolve.syncOrder({
  id: "ORDER-123", version: 1, status: "paid",
  items: [{ product_id: "CUP-01", name: "Ceramic cup",
    quantity: 1, unit_amount: "25.00" }],
});
```

Use a stable app order ID and increment version when its snapshot changes. Optional app payment references are merchant-reported until independently checked.

Add the support flow

```
// First authenticate the customer and verify ownership
// of this order in your own app database.
const orderId = "ORDER-123";
const requestId = savedComplaintUuid; // Persist once.
const receipt = await resolve.openIssue({
  orderId, requestId, reason: "damaged_item",
  message: "The cup handle arrived broken.",
  requestedResolution: "replacement",
});
const progress = await resolve.getOrderResolution(orderId);
const journey = await resolve.getRefundJourney(orderId);
const context = await resolve.getSupportContext(orderId);

// Only on an explicit action by the owning buyer:
const portal = await resolve.createSupportLink(orderId);
```

Status reads create no case and make no provider call. Opening an issue records a customer request, not an approved promise. Creating a support link replaces the previous case link, so do not rotate it while polling.

Other useful methods

Method	Purpose
lookupIssue	Recover a receipt using the original order/request UUID.
getReturnEligibility	Check items, flags and disclosed terms.
requestOrderReturn / getOrderReturns	Submit a stable request or read its workflow.
getReturnGuide / downloadReturnLabel	Show instructions and save a merchant-supplied PDF.
pullEvents / PostgresResolutionConsumer	Reconcile missed state into your app database.

- API reliability (#use-the-api-reliably)
- Download OpenAPI (../postman/contracts/resolve.openapi.json)

Use the API reliably

Choose the right authentication boundary and recover from duplicates, missed updates and timeouts.

Authentication and API families

Boundary	Purpose
Full merchant key / staff session	Merchant actions with ownership and role checks.
Support integration key	Source-granted commerce/aftercare calls from the app server.
Buyer capability	Only the case covered by the current private link.

Direct API calls use Authorization: Bearer. Website merchant sessions use HttpOnly cookies. Your app must still verify which buyer owns an order.

Path	Purpose
/resolution-cases	Promises, evidence and timelines.
/platform	Accounts, team, connections, offers, returns and packages.
/commerce/orders	Register and compare app orders.
/aftercare	Issues, status, context, return guidance and event pulls.
/cases/{id}/refund-approvals and /refund-executions	Separate financial review and submission.

Interactive local API documentation is at http://127.0.0.1:8000/docs. Use your configured backend address when hosted. The downloaded contract has exact methods, schemas and validation rules.

• Download OpenAPI (../postman/contracts/resolve.openapi.json)

Keep identity after an uncertain response

A timeout does not establish whether a request committed. Persist one UUID before sending and retain the original intent on retries. lookupIssue retrieves its receipt; reusing an ID with changed content conflicts.

Retry-safe SDK actions get up to three attempts by default within the deadline. Authentication, validation, conflicts, cancellation and long rate limits return to the caller. Portal rotation, merchant mutations and financial actions are not automatically retried. Reconcile an unknown refund rather than issuing another one.

Recover missed state

pullEvents reads a source-scoped state journal with a cursor and increasing order revisions. Pulls reconcile bounded connected orders; the operations worker also refreshes projections. Poll on a schedule for larger workspaces. This is a pull stream, not an outbound webhook subscription, and it recovers current state rather than every transient transition.

Commit the app update and cursor together

```
import { PostgresResolutionConsumer } from "@resolve-paypal/sdk";
const page = await resolve.pullEvents();
const consumer = new PostgresResolutionConsumer(pool, {
  merchantId: page.merchant_id, source: page.source,
  consumerId: "order-support",
});
await consumer.initialize();
await consumer.poll(resolve, async (event, transaction) => {
  const resolution = event.snapshot.resolution;
  if (!resolution || typeof resolution !== "object" ||
      !("stage" in resolution) ||
      typeof resolution.stage !== "string") {
    throw new Error("Missing resolution state");
  }
  await transaction.query(
    "UPDATE my_orders SET support_stage=$1 WHERE order_id=$2",
    [resolution.stage, event.order_reference],
  );
});
```

Supply your writable PostgreSQL pool and adapt the example's app table and columns. Explicit initialization creates two SDK tables. Keep a stable consumer ID and use the supplied transaction: errors roll back app writes and cursor together. Enqueue remote notifications in your own transactional outbox; external effects still need independent idempotency.

Connect supported services

Configure the owning workspace's providers and understand their current scope.

Save and check a connection

1. As an owner, open Operations -> Connections.

2. Enable the provider and enter its required configuration and secrets. Blank secret fields retain existing secrets.

3. Save and choose Check connection. Returned status is redacted.

4. Verify an actual owned record separately. Working credentials do not prove capture ownership or payment outcome.

Saved workspace credentials need a stable RESOLVE_PLATFORM_ENCRYPTION_KEY in the backend and workers. Back it up privately; replacing it can make saved secrets unreadable.

Supported scope

Service	What it does
PayPal	Sandbox payment/refund evidence and separately gated Sandbox refunds; optional merchant sign-in.
Zendesk	Ticket imports and signed webhook intake; uncertain order matches require confirmation.
Shopify / EasyPost	Optional read-only order/return/tracking evidence; postage and fulfillment remain external.
SMTP / Resend	Optional customer mail through the follow-up worker.
Google	Optional buyer sign-in for My cases.
Groq / OpenRouter	Optional advisory suggestions/explanations/investigations under configured policy.

Stripe/bank references are manual records, not live connectors. Hosted OAuth and callbacks require account-backed configuration.

Preserve support context

Enter a Zendesk ticket number under Support imports. Unique exact order matches may link a case; otherwise select and confirm a match. Source IDs and public/private labels are retained. Private imported comments are not copied into buyer conversations.

For automation, configure the displayed workspace webhook URL and matching signing secret, then run the operations worker. Hosted providers need a reachable HTTPS URL; localhost alone cannot receive callbacks.

Set a customer-visible sender

```
RESOLVE_CUSTOMER_EMAIL_ENABLED=false
RESOLVE_CUSTOMER_EMAIL_TRANSPORT=smtp
RESOLVE_SMTP_HOST=<hostname-from-your-email-provider>
RESOLVE_SMTP_PORT=587
RESOLVE_SMTP_SECURE=false
RESOLVE_SMTP_USER=<your-smtp-login>
RESOLVE_SMTP_PASSWORD=<your-smtp-credential>
RESOLVE_SMTP_FROM=Resolve Support <support@example.com>
```

Use a sender your provider authorizes. SMTP_USER is the login; SMTP_FROM is the visible sender. Follow your provider's port/TLS requirements. Enable delivery after configuring mail and the required merchant binding. Saving settings does not send mail; the follow-up worker performs delivery.

Run and configure Resolve

Start a responsive local preview and keep settings and workers in the correct environment.

Prerequisites

• Python 3.12 or newer and uv.

• Node.js 22.12 or newer for the SDK, which also satisfies the frontend requirement.

• SQLite for a local start, or explicitly configured PostgreSQL.

• Provider credentials only for integrations you enable.

Use ignored local files or a deployment secret store. Examples contain placeholders. Preserve your existing database settings when preparing a separate test environment.

Start the API and optimized website

```
# Terminal 1, from the repository root
Set-Location backend
uv sync
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000

# Terminal 2, from the repository root
npm --prefix frontend ci
npm --prefix frontend run build:preview
npm --prefix frontend run preview
```

Open http://127.0.0.1:3000/account and keep both processes running. Stop another frontend on port 3000 first. The compiled preview uses .next-preview; rebuild and restart after source/configuration changes. Use npm --prefix frontend run dev for automatic reload while editing, instead of the compiled preview.

Configuration locations

Location	Purpose
backend/.env	Database, ordinary providers, email, origins and switches. See backend/.env.example.
frontend/.env.local	Backend URL and website origins. See frontend/.env.example.
Private PowerShell AI environment	Copy backend/ai-env.example.ps1 and explicitly load the private copy into API/worker processes.
Merchant app server	RESOLVE_API_URL, RESOLVE_API_KEY and RESOLVE_APP_SOURCE.

AI settings use process variables rather than ordinary dotenv loading. Enable only configured features. Keep browser origins consistent and hosted callbacks on HTTPS.

Run only the workers you need

Command from backend	Work
uv run python -m app.operations_worker --loop	Imports, overdue scans and projections; no mail or financial submission.
uv run python -m app.investigation_worker --loop	Automatic evidence investigations under configured policy.
uv run python -m app.followup_worker --loop	Customer follow-up jobs and optional email delivery.
uv run python -m app.worker --loop	Configured refund recovery/reconciliation; review its CLI/payment settings first.

Workers need the correct database, encryption key and environment. The website does not start them. Use --help for interval/limit controls. Keep numbered migrations immutable and test schema changes against disposable storage. Hosted operation also needs stable origins, backups, secret management and account-backed checks.

Troubleshoot common problems

Find a useful next check for login, returns, refunds, background jobs and preview updates.

Login and support access

Symptom	Next check
Cannot sign in	Use an individual account or full merchant key. Commerce/support keys cannot sign in.
Empty workspace	Check the merchant account; newly created workspaces have separate records.
SDK 403	Verify app source and explicit support grant; order-only keys cannot use aftercare.
Invalid buyer link	Ask for a current link; expired, replaced and revoked links cannot be reused.

Orders, returns and evidence

Symptom	Next check
No return items	Register owned items, connect the order to the case and check existing reservations.
Stale return review	Reload current policy/return versions and record a current review or permitted exception.
No saved label	The merchant must arrange and upload a PDF; postage is external.
Stale evidence export	Build a fresh package, inspect sources and review before export.

Refund progress and timeouts

Symptom	Next check
Unknown amounts	Refresh evidence; unavailable/stale proof is not zero.
Approval blocked	Read acceptance/conditions, provider history, balance, pending refunds, disputes and uncertain executions.
Processor complete, bank money missing	Check provider proof and any merchant tracing reference. Bank receipt is not independently verified.
Network timeout	Retain the original UUID and recover/reconcile that request instead of creating another refund.

Workers and AI

Symptom	Next check
Imports or alerts not updating	Check the operations worker, connection status and owning database/key.
Investigation queued	Run its worker and inspect status. Missing advice does not authorize anything.
No email	Check enabled transport, sender, merchant binding, queue and follow-up worker.
AI fallback	Check process credentials, switches, policy, routes and budgets; abstention can be valid.

Website updates

1. Use the compiled preview for normal viewing; a development route may compile before loading.

2. After a frontend edit or merge, rebuild with npm --prefix frontend run build:preview and restart npm --prefix frontend run preview.

3. Refresh with Ctrl + Shift + R for new assets, and refresh messages/details for newly arrived data.

4. Check http://127.0.0.1:8000/health and RESOLVE_BACKEND_URL if the API is unreachable. Hosted database network latency still affects reads.

- Run Resolve (#run-and-configure-resolve)
- API reliability (#use-the-api-reliably)